01 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() accumulate elements

filter() null checks + collect() accumulate elements

filter() comparisons + collect() accumulate elements

filter() method calls returning boolean + collect() accumulate elements

distinct() rely on equals() / hashcode() + collect() accumulate elements

distinct() immutable objects + collect() accumulate elements

map() transform objects + collect() accumulate elements

map() create new objects + collect() accumulate elements

map() call pure functions + collect() accumulate elements

flatmap() return stream + collect() accumulate elements

flatmap() split data + collect() accumulate elements

flatmap() expand collections + collect() accumulate elements

sorted() natural ordering + collect() accumulate elements

sorted() custom comparator + collect() accumulate elements

sorted() multi-field compare + collect() accumulate elements

peek() logging + collect() accumulate elements

peek() debugging + collect() accumulate elements

peek() metrics + collect() accumulate elements

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, self-contained Java class examples** for **every requested Stream combination**, each focusing on **one intermediate operation + collect()**.

---

1 filter()— boolean checks + collect()

```
import java.util.*;

public class FilterBooleanCheckExample {
   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(10, 15, 20, 25, 30);
      List < Integer > evenNumbers = new ArrayList < Integer > ();

      for (Integer n: numbers) {
         if (n % 2 == 0) {
            evenNumbers.add(n);
         }
      }

      System.out.println(evenNumbers);
   }
}
```

---

2 filter()— null checks + collect()

```
import java.util.*;

public class FilterNullCheckExample {
   public static void main(String[] args) {

      List < String > names = Arrays.asList("Anurag", null, "Bharti", null);
      List < String > nonNullNames = new ArrayList < String > ();

      for (String name: names) {
         if (name != null) {
            nonNullNames.add(name);
         }
      }

      System.out.println(nonNullNames);
   }
}
```

---

3 filter()— comparisons + collect()

```
import java.util.*;

public class FilterComparisonExample {
   public static void main(String[] args) {

      List < Integer > salaries = Arrays.asList(20000, 35000, 50000);
      List < Integer > highSalaries = new ArrayList < Integer > ();

      for (Integer salary: salaries) {
         if (salary > 30000) {
            highSalaries.add(salary);
         }
      }

      System.out.println(highSalaries);
   }
}
```

---

4 filter()— method calls returning boolean + collect()

```
import java.util.*;

public class FilterMethodCallExample {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List < User > users = new ArrayList < User > ();
      users.add(new User("Anurag", true));
      users.add(new User("Ravi", false));

      List < User > activeUsers = new ArrayList < User > ();

      for (User user: users) {
         if (user.isActive()) {
            activeUsers.add(user);
         }
      }

      System.out.println(activeUsers.size());
   }
}
```

---

5 distinct()— rely on equals() / hashCode() + collect()

```
import java.util.*;

public class DistinctWithEqualsExample {

   static class Person {
      String name;

      Person(String name) {
         this.name = name;
      }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List < Person > people = new ArrayList < Person > ();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set < Person > set = new LinkedHashSet < Person > (people);
      List < Person > unique = new ArrayList < Person > (set);

      System.out.println(unique.size());
   }
}
```

---

6 distinct()— immutable objects + collect()

```
import java.util.*;

public class DistinctImmutableExample {
   public static void main(String[] args) {

      List < String > values = Arrays.asList("A", "B", "A", "C");

      Set < String > set = new LinkedHashSet < String > (values);
      List < String > unique = new ArrayList < String > (set);

      System.out.println(unique);
   }
}
```

---

7 map()— transform objects + collect()

```
import java.util.*;

public class MapTransformExample {
   public static void main(String[] args) {

      List < String > names = Arrays.asList("anurag", "bharti");
      List < String > upper = new ArrayList < String > ();

      for (String name: names) {
         upper.add(name.toUpperCase());
      }

      System.out.println(upper);
   }
}
```

---

8 map()— create new objects + collect()

```
import java.util.*;

public class MapCreateObjectExample {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {

      List < String > names = Arrays.asList("Anurag", "Ravi");
      List < UserDTO > dtos = new ArrayList < UserDTO > ();

      for (String name: names) {
         dtos.add(new UserDTO(name));
      }

      System.out.println(dtos.size());
   }
}
```

---

9 map()— call pure functions + collect()

```
import java.util.*;

public class MapPureFunctionExample {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(2, 3, 4);
      List < Integer > squares = new ArrayList < Integer > ();

      for (Integer n: numbers) {
         squares.add(square(n));
      }
      System.out.println(squares);
```

```
    }
}
```

---

10 flatMap() — return Stream + collect()

```
import java.util.*;

public class FlatMapStreamExample {
    public static void main(String[] args) {

        List < List < Integer >> lists = new ArrayList < List < Integer >> ();
        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        List < Integer > result = new ArrayList < Integer > ();

        for (List < Integer > inner: lists) {
            result.addAll(inner);
        }

        System.out.println(result);
    }
}
```

---

11 flatMap()— split data + collect()

```
import java.util.*;

public class FlatMapSplitExample {
    public static void main(String[] args) {

        List < String > sentences = Arrays.asList("Java Stream", "Flat Map");
        List < String > words = new ArrayList < String > ();

        for (String s: sentences) {
            String[] split = s.split(" ");
            for (String word: split) {
                words.add(word);
            }
        }

        System.out.println(words);
    }
}
```

---

12 flatMap()— expand collections + collect()

```
import java.util.*;

public class FlatMapExpandExample {
   public static void main(String[] args) {

      Map < String, List < String >> data = new HashMap < String, List < String >> ();
      data.put("A", Arrays.asList("1", "2"));
      data.put("B", Arrays.asList("3"));

      List < String > values = new ArrayList < String > ();

      for (List < String > list: data.values()) {
         values.addAll(list);
      }

      System.out.println(values);
   }
}
```

---

13 sorted()— natural ordering + collect()

```
import java.util.*;

public class SortedNaturalExample {
   public static void main(String[] args) {

      List < Integer > numbers = new ArrayList < Integer > (
         Arrays.asList(5, 1, 3));

      Collections.sort(numbers);

      System.out.println(numbers);
   }
}
```

---

14 sorted()— custom comparator + collect()

```
import java.util.*;

public class SortedCustomComparatorExample {
   public static void main(String[] args) {

      List < String > names = new ArrayList < String > (
         Arrays.asList("Anurag", "Ravi", "Alok"));

      Collections.sort(names, Collections.reverseOrder());

      System.out.println(names);
   }
}
```

---

15 sorted()— multi - field compare + collect()

```
import java.util.*;

public class SortedMultiFieldFindFirst {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;
            return e1.age - e2.age;
         }
      });
```

```
        Employee first = list.size() > 0 ? list.get(0) : null;

        System.out.println(first != null ? first.name + " " + first.age : "NONE");
    }
}
```

---

16 peek()— logging + collect()

```
import java.util.*;

public class PeekLoggingExample {
    public static void main(String[] args) {

        List < Integer > numbers = Arrays.asList(1, 2, 3);
        List < Integer > result = new ArrayList < Integer > ();

        for (Integer n: numbers) {
            System.out.println("Processing: " + n);
            result.add(n);
        }

        System.out.println(result);
    }
}
```

---

17 peek()— debugging + collect()

```
import java.util.*;

public class PeekDebugExample {
    public static void main(String[] args) {

        List < Integer > numbers = Arrays.asList(10, 20, 30);
        List < Integer > result = new ArrayList < Integer > ();

        for (Integer n: numbers) {
            if (n == 20) {
                System.out.println("Breakpoint value: " + n);
            }
            result.add(n);
        }

        System.out.println(result);
    }
}
```

---

18 peek()— metrics + collect()

```
import java.util.*;

public class PeekMetricsExample {
   public static void main(String[] args) {

      List < Integer > numbers = Arrays.asList(5, 10, 15);
      List < Integer > result = new ArrayList < Integer > ();

      int counter = 0;

      for (Integer n: numbers) {
         counter++;
         result.add(n);
      }

      System.out.println("Processed count: " + counter);
   }
}
```

---

02 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() transform & group

filter() null checks + collect() transform & group

filter() comparisons + collect() transform & group

filter() method calls returning boolean + collect() transform & group

distinct() rely on equals() / hashcode() + collect() transform & group

distinct() immutable objects + collect() transform & group

map() transform objects + collect() transform & group

map() create new objects + collect() transform & group

map() call pure functions + collect() transform & group

flatmap() return stream + collect() transform & group

flatmap() split data + collect() transform & group

flatmap() expand collections + collect() transform & group

sorted() natural ordering + collect() transform & group

sorted() custom comparator + collect() transform & group

sorted() multi-field compare + collect() transform & group

peek() logging + collect() transform & group

peek() debugging + collect() transform & group

peek() metrics + collect() transform & group

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Perfect — this time **every example explicitly does BOTH**:

1. **Transform** (via map, mapping, or downstream collectors)
2. **Group** (via Collectors.groupingBy(...))

Each item is a **complete, standalone Java class**, Java 8+, clean and interview-ready.

---

1 filter() boolean checks + collect() (Transform & Group)

```
import java.util.*;


public class FilterBooleanTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);
      Map<String, List<Integer>> result = new HashMap<String, List<Integer>>();

      for (Integer n : numbers) {

         if (n % 2 == 0) {                                    // filter
            int transformed = n * 2;                          // map

            String key = transformed > 30 ? "HIGH" : "LOW";

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<Integer>());
            }
            result.get(key).add(transformed);
         }
      }

      System.out.println(result);
   }
}
```

---

2 filter() null checks + collect() (Transform & Group)

```
import java.util.*;

public class FilterNullTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", null, "Amit", null, "Bharti");
      Map<Integer, List<String>> result = new HashMap<Integer, List<String>>();

      for (String name : names) {

         if (name != null) {                                    // filter null
            String upper = name.toUpperCase();                  // map
            Integer key = upper.length();

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<String>());
            }
            result.get(key).add(upper);
         }
      }

      System.out.println(result);
   }
}
```

---

3 filter() comparisons + collect() (Transform & Group)

```
import java.util.*;

public class FilterComparisonTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> salaries = Arrays.asList(20000, 40000, 60000);
      Map<String, List<Integer>> result = new HashMap<String, List<Integer>>();

      for (Integer s : salaries) {

         if (s >= 30000) {
            int transformed = s + 5000;

            String key = transformed >= 50000 ? "SENIOR" : "MID";

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<Integer>());
            }
            result.get(key).add(transformed);
         }
      }

      System.out.println(result);
   }
}
```

---

4 filter() method call boolean + collect() (Transform & Group)

```
import java.util.*;

public class FilterMethodTransformGroup_Java15 {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List<User> users = Arrays.asList(
            new User("Anurag", true),
            new User("Ravi", false),
            new User("Amit", true)
      );

      Map<Boolean, List<String>> result = new HashMap<Boolean, List<String>>();

      for (User u : users) {

         if (u.isActive()) {                                    // filter
            String upper = u.name.toUpperCase();                // map
            Boolean key = upper.startsWith("A");

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<String>());
            }
            result.get(key).add(upper);
         }
      }

      System.out.println(result);
   }
}
```

---

5 distinct() equals/hashCode + collect() (Transform & Group)

```
import java.util.*;

public class DistinctEqualsTransformGroup_Java15 {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;
         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = Arrays.asList(
            new Person("Anurag"),
            new Person("Anurag"),
            new Person("Ravi")
      );

      Set<Person> unique = new HashSet<Person>(people);
      Map<Integer, List<String>> result = new HashMap<Integer, List<String>>();

      for (Person p : unique) {

         String name = p.name;
         Integer key = name.length();

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<String>());
         }
         result.get(key).add(name);
      }

      System.out.println(result);
```

```
    }
}
```

---

6 distinct() immutable objects + collect() (Transform & Group)

```
import java.util.*;

public class DistinctImmutableTransformGroup_Java15 {
    public static void main(String[] args) {

        List<String> values = Arrays.asList("Java", "Java", "Python", "Go");

        Set<String> unique = new HashSet<String>(values);
        Map<Character, List<String>> result = new HashMap<Character, List<String>>();

        for (String s : unique) {

            String upper = s.toUpperCase();
            Character key = upper.charAt(0);

            if (!result.containsKey(key)) {
                result.put(key, new ArrayList<String>());
            }
            result.get(key).add(upper);
        }

        System.out.println(result);
    }
}
```

---

7 map() transform objects + collect() (Transform & Group)

```
import java.util.*;

public class MapTransformGroup_Java15 {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("anurag", "amit", "ravi");
        Map<Integer, List<String>> result = new HashMap<Integer, List<String>>();

        for (String name : names) {

            String upper = name.toUpperCase();
            Integer key = upper.length();

            if (!result.containsKey(key)) {
                result.put(key, new ArrayList<String>());
```

```
        }
        result.get(key).add(upper);
    }

    System.out.println(result);
  }
}
```

---

8 map() create new objects + collect() (Transform & Group)

```
import java.util.*;

public class MapNewObjectTransformGroup_Java15 {

  static class UserDTO {
    String name;

    UserDTO(String name) {
      this.name = name;
    }
  }

  public static void main(String[] args) {

    List<String> names = Arrays.asList("Anurag", "Amit", "Ravi");
    Map<Character, List<UserDTO>> result =
        new HashMap<Character, List<UserDTO>>();

    for (String name : names) {

      UserDTO dto = new UserDTO(name);                    // map → new object
      Character key = dto.name.charAt(0);

      if (!result.containsKey(key)) {
        result.put(key, new ArrayList<UserDTO>());
      }
      result.get(key).add(dto);
    }

    System.out.println(result);
  }
}
```

---

9 map() pure functions + collect() (Transform & Group)

```
import java.util.*;

public class MapPureFunctionTransformGroup_Java15 {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(2, 3, 4);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         int squared = square(n);                                    // map
         Boolean key = squared > 10;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(squared);
      }

      System.out.println(result);
   }
}
```

---

10 flatMap() return Stream + collect() (Transform & Group)

```
import java.util.*;

public class FlatMapStreamTransformGroup_Java15 {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (List<Integer> inner : lists) {                    // flatMap → nested loop
         for (Integer n : inner) {

            int transformed = n * 10;                        // map
            Boolean key = transformed > 20;

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<Integer>());
            }
            result.get(key).add(transformed);
         }
      }
      System.out.println(result);
   }
}
```

---

11 flatMap() split data + collect() (Transform & Group)

```
import java.util.*;

public class FlatMapSplitTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> sentences =
            Arrays.asList("Java Stream", "Flat Map");

      Map<Integer, List<String>> result =
            new HashMap<Integer, List<String>>();

      for (String sentence : sentences) {

         String[] words = sentence.split(" ");                    // split

         for (String word : words) {                              // flatMap
            String lower = word.toLowerCase();
            Integer key = lower.length();

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<String>());
            }
            result.get(key).add(lower);
         }
      }

      System.out.println(result);
   }
}
```

12 flatMap() expand collections + collect() (Transform & Group)

```
import java.util.*;

public class FlatMapExpandTransformGroup_Java15 {
   public static void main(String[] args) {

      Map<String, List<String>> data =
            new HashMap<String, List<String>>();

      data.put("A", Arrays.asList("x", "y"));
      data.put("B", Arrays.asList("z"));

      Map<Integer, List<String>> result =
            new HashMap<Integer, List<String>>();

      for (List<String> values : data.values()) {

         for (String value : values) {                          // flatMap
            String upper = value.toUpperCase();
            Integer key = upper.length();

            if (!result.containsKey(key)) {
               result.put(key, new ArrayList<String>());
            }
            result.get(key).add(upper);
         }
      }

      System.out.println(result);
   }
}
```

---

13 sorted() natural order + collect() (Transform & Group)

```
import java.util.*;

public class SortedNaturalTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers =
            new ArrayList<Integer>(Arrays.asList(5, 1, 3));

      Collections.sort(numbers);                                    // natural sort

      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         int transformed = n * 2;
         Boolean key = transformed > 5;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
      }

      System.out.println(result);
   }
}
```

14 sorted() custom comparator + collect() (Transform & Group)

```
import java.util.*;

public class SortedCustomTransformGroup_Java15 {
   public static void main(String[] args) {

      List<String> names =
            new ArrayList<String>(Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(names, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                                  // reverse order
         }
      });

      Map<Character, List<String>> result =
            new HashMap<Character, List<String>>();

      for (String name : names) {

         String upper = name.toUpperCase();
         Character key = upper.charAt(0);

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<String>());
         }
         result.get(key).add(upper);
      }
      System.out.println(result);
   }
}
```

---

15 sorted() multi-field + collect() (Transform & Group)

```
import java.util.*;

public class SortedMultiFieldTransformGroup_Java15 {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> employees =
            new ArrayList<Employee>(Arrays.asList(
                  new Employee("Anurag", 30),
                  new Employee("Anurag", 25),
                  new Employee("Ravi", 35)
            ));

      Collections.sort(employees, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0)
               return nameCompare;
            return e1.age - e2.age;
         }
      });

      Map<String, List<Integer>> result =
            new HashMap<String, List<Integer>>();

      for (Employee e : employees) {

         Integer age = e.age;
         String key = age >= 30 ? "SENIOR" : "JUNIOR";

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(age);
```

```
    }
    System.out.println(result);
  }
}
```

---

16 peek() logging + collect() (Transform & Group)

```
import java.util.*;

public class PeekLoggingTransformGroup_Java15 {
  public static void main(String[] args) {

    List<Integer> numbers = Arrays.asList(1, 2, 3);
    Map<Boolean, List<Integer>> result =
        new HashMap<Boolean, List<Integer>>();

    for (Integer n : numbers) {

      System.out.println("Log: " + n);                    // peek equivalent

      int transformed = n * 10;
      Boolean key = transformed > 20;

      if (!result.containsKey(key)) {
        result.put(key, new ArrayList<Integer>());
      }
      result.get(key).add(transformed);
    }

    System.out.println(result);
  }
}
```

---

17 peek() debugging + collect() (Transform & Group)

```
import java.util.*;

public class PeekDebugTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 20, 30);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      for (Integer n : numbers) {

         if (n == 20) {
            System.out.println("Debug hit: " + n);
         }

         int transformed = n + 1;
         Boolean key = transformed > 20;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
      }

      System.out.println(result);
   }
}
```

---

18 peek() metrics + collect() (Transform & Group)

```
import java.util.*;

public class PeekMetricsTransformGroup_Java15 {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(5, 10, 15);
      Map<Boolean, List<Integer>> result =
            new HashMap<Boolean, List<Integer>>();

      int counter = 0;                          // AtomicInteger not needed in Java 1.5 single-thread

      for (Integer n : numbers) {

         counter++;                                                    // peek metric

         int transformed = n * 2;
         Boolean key = transformed > 20;

         if (!result.containsKey(key)) {
            result.put(key, new ArrayList<Integer>());
         }
         result.get(key).add(transformed);
      }
      System.out.println("Processed: " + counter);
      System.out.println(result);
   }
}
```

---

03 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + collect() map → reduce → store

filter() null checks + collect() map → reduce → store

filter() comparisons + collect() map → reduce → store

filter() method calls returning boolean + collect() map → reduce → store

distinct() rely on equals() / hashcode() + collect() map → reduce → store

distinct() immutable objects + collect() map → reduce → store

map() transform objects + collect() map → reduce → store

map() create new objects + collect() map → reduce → store

map() call pure functions + collect() map → reduce → store

flatmap() return stream + collect() map → reduce → store

flatmap() split data + collect() map → reduce → store

flatmap() expand collections + collect() map → reduce → store

sorted() natural ordering + collect() map → reduce → store

sorted() custom comparator + collect() map → reduce → store

sorted() multi-field compare + collect() map → reduce → store

peek() logging + collect() map → reduce → store

peek() debugging + collect() map → reduce → store

peek() metrics + collect() map → reduce → store

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline explicitly follows**:

Map → Reduce → Store

- **Map** = transform data
- **Reduce** = aggregate (sum / count / max / min / join / etc.)
- **Store** = collect() into a Map or List

All examples:

- Java **8+**
- One class = one concept
- Interview-safe and production-realistic
- No shortcuts, no magic

---

1 filter() boolean checks + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterBooleanMapReduceStore {
   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);
      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Integer n : numbers) {

         if (n % 2 == 0) {                                    // FILTER

            int mapped = n * 2;                               // MAP

            String key = (mapped > 30) ? "HIGH" : "LOW";

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                    // REDUCE + STORE
         }
      }

      System.out.println(result);
```

```
    }
}
```

---

2 filter() null checks + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterNullMapReduceStore {
    public static void main(String[] args) {

        List<String> names = Arrays.asList("Anurag", null, "Amit", null);
        Map<Integer, Long> result = new HashMap<Integer, Long>();

        for (String name : names) {

            if (name != null) {                                  // FILTER

                String mapped = name.toUpperCase();              // MAP
                Integer key = mapped.length();

                Long count = result.get(key);
                if (count == null) count = 0L;

                result.put(key, count + 1);                      // REDUCE + STORE
            }
        }

        System.out.println(result);
    }
}
```

---

3 filter() comparisons + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterComparisonMapReduceStore {
   public static void main(String[] args) {

      List<Integer> salaries = Arrays.asList(20000, 40000, 60000);
      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Integer s : salaries) {

         if (s >= 30000) {                                        // FILTER

            int mapped = s + 5000;                                // MAP
            String key = (mapped >= 50000) ? "SENIOR" : "MID";

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                        // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

4 filter() method call boolean + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FilterMethodMapReduceStore {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List<User> users = Arrays.asList(
            new User("Anurag", true),
            new User("Amit", true),
            new User("Ravi", false)
      );

      Map<Character, Long> result = new HashMap<Character, Long>();

      for (User u : users) {

         if (u.isActive()) {                                    // FILTER

            String mapped = u.name;                             // MAP
            Character key = mapped.charAt(0);

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                         // REDUCE + STORE
         }
      }
```

```
        System.out.println(result);
    }
}
```

---

5 distinct() equals/hashCode + collect() (Map → Reduce → Store)

```
import java.util.*;

public class DistinctEqualsMapReduceStore {

    static class Person {
        String name;

        Person(String name) { this.name = name; }

        public boolean equals(Object o) {
            if (o instanceof Person) {
                Person p = (Person) o;
                return name.equals(p.name);
            }
            return false;
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> people = Arrays.asList(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        Set<Person> distinct = new HashSet<Person>(people);

        Map<Integer, Long> result = new HashMap<Integer, Long>();

        for (Person p : distinct) {

            String mapped = p.name;                                    // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;
```

```
        result.put(key, count + 1);                    // REDUCE + STORE
    }

    System.out.println(result);
  }
}
```

---

6 distinct() immutable objects + collect() (Map → Reduce → Store)

```
import java.util.*;

public class DistinctImmutableMapReduceStore {

  public static void main(String[] args) {

    // Java 1.5 way to create list
    List<String> values = new ArrayList<String>();
    values.add("Java");
    values.add("Java");
    values.add("Go");
    values.add("Python");

    // DISTINCT step
    Set<String> distinctSet = new HashSet<String>(values);

    // MAP + REDUCE + STORE
    Map<Character, Long> result = new HashMap<Character, Long>();

    for (String value : distinctSet) {

      // MAP (toUpperCase)
      String upper = value.toUpperCase();

      // key = first character
      Character key = upper.charAt(0);

      // REDUCE (counting)
      Long count = result.get(key);

      if (count == null) {
        result.put(key, 1L);
      } else {
        result.put(key, count + 1);
      }
    }

    System.out.println(result);
  }
}
```

---

7 map() transform objects + collect() (Map → Reduce → Store)

```
import java.util.*;

public class MapTransformMapReduceStore {
   public static void main(String[] args) {

      List<String> names = Arrays.asList("anurag", "amit", "ravi");
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String name : names) {

         String mapped = name.toUpperCase();                    // MAP
         Integer key = mapped.length();

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                            // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

8 map() create new objects + collect() (Map → Reduce → Store)

```
import java.util.*;

public class MapNewObjectMapReduceStore {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> names = Arrays.asList("Anurag", "Amit", "Ravi");
      Map<Character, Long> result = new HashMap<Character, Long>();

      for (String name : names) {
```

```
        UserDTO dto = new UserDTO(name);                    // MAP
        Character key = dto.name.charAt(0);

        Long count = result.get(key);
        if (count == null) count = 0L;

        result.put(key, count + 1);                         // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

9 map() pure functions + collect() (Map → Reduce → Store)

```
import java.util.*;

public class MapPureFunctionMapReduceStore {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = Arrays.asList(2, 3, 4);
      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : numbers) {

        int mapped = square(n);                             // MAP
        Boolean key = (mapped > 10);

        Integer sum = result.get(key);
        if (sum == null) sum = 0;

        result.put(key, sum + mapped);                      // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

10 flatMap() return Stream + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FlatMapStreamMapReduceStore {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();
      lists.add(Arrays.asList(1, 2));
      lists.add(Arrays.asList(3, 4));

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (List<Integer> inner : lists) {                    // FLATTEN
         for (Integer n : inner) {

            int mapped = n * 10;                             // MAP
            Boolean key = (mapped > 20);

            Integer sum = result.get(key);
            if (sum == null) sum = 0;

            result.put(key, sum + mapped);                   // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

11 flatMap() split data + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FlatMapSplitMapReduceStore {
   public static void main(String[] args) {

      List<String> sentences = Arrays.asList("Java Stream", "Flat Map");
      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (String sentence : sentences) {

         String[] words = sentence.split(" ");                    // SPLIT

         for (int i = 0; i < words.length; i++) {

            String mapped = words[i].toLowerCase();               // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                           // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

12 flatMap() expand collections + collect() (Map → Reduce → Store)

```
import java.util.*;

public class FlatMapExpandMapReduceStore {
   public static void main(String[] args) {

      Map<String, List<String>> data = new HashMap<String, List<String>>();
      data.put("A", Arrays.asList("x", "y"));
      data.put("B", Arrays.asList("z"));

      Map<Integer, Long> result = new HashMap<Integer, Long>();

      for (List<String> list : data.values()) {                    // FLATTEN
         for (String value : list) {

            String mapped = value.toUpperCase();                    // MAP
            Integer key = mapped.length();

            Long count = result.get(key);
            if (count == null) count = 0L;

            result.put(key, count + 1);                             // REDUCE + STORE
         }
      }

      System.out.println(result);
   }
}
```

---

13 sorted() natural order + collect() (Map → Reduce → Store)

```
import java.util.*;

public class SortedNaturalMapReduceStore {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>(
            Arrays.asList(5, 1, 3));

      Collections.sort(numbers);                                  // SORT

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : numbers) {

         int mapped = n * 2;                                      // MAP
         Boolean key = (mapped > 5);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                           // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

14 sorted() custom comparator + collect() (Map → Reduce → Store)

```
import java.util.*;

public class SortedCustomMapReduceStore {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(names, Collections.reverseOrder());    // CUSTOM SORT

      Map<Character, Long> result = new HashMap<Character, Long>();

      for (String name : names) {

         String mapped = name.toUpperCase();                  // MAP
         Character key = mapped.charAt(0);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                          // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

15 sorted() multi-field + collect() (Map → Reduce → Store)

```
import java.util.*;

public class SortedMultiFieldMapReduceStore {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> employees = new ArrayList<Employee>();
      employees.add(new Employee("Anurag", 30));
      employees.add(new Employee("Anurag", 25));
      employees.add(new Employee("Ravi", 35));

      Collections.sort(employees, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {

            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;

            return e1.age - e2.age;
         }
      });

      Map<String, Integer> result = new HashMap<String, Integer>();

      for (Employee e : employees) {

         int mapped = e.age;                                    // MAP
         String key = (mapped >= 30) ? "SENIOR" : "JUNIOR";

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                         // REDUCE + STORE
```

```
    }

    System.out.println(result);
  }
}
```

---

16 peek() logging + collect() (Map → Reduce → Store)

```
import java.util.*;

public class PeekLoggingMapReduceStore {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(1, 2, 3);
    Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

    for (Integer n : list) {

      System.out.println("Log: " + n);                    // PEEK

      int mapped = n * 10;                                // MAP
      Boolean key = (mapped > 20);

      Integer sum = result.get(key);
      if (sum == null) sum = 0;

      result.put(key, sum + mapped);                      // REDUCE + STORE
    }

    System.out.println(result);
  }
}
```

---

17 peek() debugging + collect() (Map → Reduce → Store)

```
import java.util.*;

public class PeekDebugMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      Map<Boolean, Long> result = new HashMap<Boolean, Long>();

      for (Integer n : list) {

         if (n == 20) {
            System.out.println("Debug hit: " + n);                    // PEEK
         }

         int mapped = n + 1;                                          // MAP
         Boolean key = (mapped > 20);

         Long count = result.get(key);
         if (count == null) count = 0L;

         result.put(key, count + 1);                                  // REDUCE + STORE
      }

      System.out.println(result);
   }
}
```

---

18 peek() metrics + collect() (Map → Reduce → Store)

```
import java.util.*;

public class PeekMetricsMapReduceStore {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);
      int counter = 0;

      Map<Boolean, Integer> result = new HashMap<Boolean, Integer>();

      for (Integer n : list) {

         counter++;                                        // METRICS (peek equivalent)

         int mapped = n * 2;                               // MAP
         Boolean key = (mapped > 20);

         Integer sum = result.get(key);
         if (sum == null) sum = 0;

         result.put(key, sum + mapped);                    // REDUCE + STORE
      }

      System.out.println("Processed: " + counter);
      System.out.println(result);
   }
}
```

---

04 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() logging

filter() null checks + foreach() logging

filter() comparisons + foreach() logging

filter() method calls returning boolean + foreach() logging

distinct() rely on equals() / hashcode() + foreach() logging

distinct() immutable objects + foreach() logging

map() transform objects + foreach() logging

map() create new objects + foreach() logging

map() call pure functions + foreach() logging

flatmap() return stream + foreach() logging

flatmap() split data + foreach() logging

flatmap() expand collections + foreach() logging

sorted() natural ordering + foreach() logging

sorted() custom comparator + foreach() logging

sorted() multi-field compare + foreach() logging

peek() logging + foreach() logging

peek() debugging + foreach() logging

peek() metrics + foreach() logging

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline ends with forEach() used purely for logging**.

Key rules followed:

- **One intermediate concept per class**
- **forEach() is the terminal operation**
- Logging is explicit (System.out.println)
- Java **8+**, interview-safe
- No collectors, no shortcuts

---

1 filter() boolean checks + forEach() Logging

```
import java.util.*;

public class FilterBooleanForEachLogging {
   public static void main(String[] args) {
      List<Integer> numbers = Arrays.asList(10, 15, 20, 25);

      for (Integer n : numbers) {
         if (n % 2 == 0) {
            System.out.println("Even: " + n);
         }
      }
   }
}
```

---

2 filter() null checks + forEach() Logging

```
import java.util.*;

public class FilterNullForEachLogging {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      for (String name : names) {
         if (name != null) {
```

```
            System.out.println("Name: " + name);
        }
    }
  }
}
```

---

3 filter() comparisons + forEach() Logging

```
import java.util.*;

public class FilterComparisonForEachLogging {
  public static void main(String[] args) {
    List<Integer> salaries = Arrays.asList(20000, 40000, 60000);

    for (Integer s : salaries) {
      if (s >= 30000) {
        System.out.println("Eligible salary: " + s);
      }
    }
  }
}
```

---

4 filter() method call boolean + forEach() Logging

```
import java.util.*;

public class FilterMethodForEachLogging {

  static class User {
    String name;
    boolean active;

    User(String name, boolean active) {
      this.name = name;
      this.active = active;
    }

    boolean isActive() {
      return active;
    }
  }

  public static void main(String[] args) {
    List<User> users = Arrays.asList(
        new User("Anurag", true),
        new User("Ravi", false)
```

```
        );

        for (User u : users) {
            if (u.isActive()) {
                System.out.println("Active user: " + u.name);
            }
        }
    }
}
```

---

5 distinct() equals/hashCode + forEach() Logging

```
import java.util.*;

public class DistinctEqualsForEachLogging {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (o instanceof Person) {
                Person p = (Person) o;
                return name.equals(p.name);
            }
            return false;
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {
        List<Person> people = Arrays.asList(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        Set<Person> unique = new HashSet<Person>(people);

        for (Person p : unique) {
            System.out.println("Unique person: " + p.name);
        }
```

```
    }
}
```

---

6 distinct() immutable objects + forEach() Logging

```
import java.util.*;

public class DistinctImmutableForEachLogging {
    public static void main(String[] args) {
        List<String> languages = Arrays.asList("Java", "Java", "Python");

        Set<String> unique = new HashSet<String>(languages);

        for (String lang : unique) {
            System.out.println("Language: " + lang);
        }
    }
}
```

---

7 map() transform objects + forEach() Logging

```
import java.util.*;

public class MapTransformForEachLogging {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("anurag", "amit");

        for (String name : names) {
            String upper = name.toUpperCase();
            System.out.println("Upper: " + upper);
        }
    }
}
```

---

8 map() create new objects + forEach() Logging

```
import java.util.*;

public class MapNewObjectForEachLogging {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", "Amit");

      for (String name : names) {
         UserDTO dto = new UserDTO(name);
         System.out.println("DTO created: " + dto.name);
      }
   }
}
```

---

9 map() pure functions + forEach() Logging

```
import java.util.*;

public class MapPureFunctionForEachLogging {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = Arrays.asList(2, 3, 4);

      for (Integer n : numbers) {
         int result = square(n);
         System.out.println("Square: " + result);
```

```
        }
    }
}
```

---

10 flatMap() return Stream + forEach() Logging

```
import java.util.*;

public class FlatMapStreamForEachLogging {
    public static void main(String[] args) {
        List<List<Integer>> lists = new ArrayList<List<Integer>>();
        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        for (List<Integer> inner : lists) {
            for (Integer n : inner) {
                System.out.println("Value: " + n);
            }
        }
    }
}
```

---

11 flatMap() split data + forEach() Logging

```
import java.util.*;

public class FlatMapSplitForEachLogging {
    public static void main(String[] args) {
        List<String> sentences = Arrays.asList("Java Stream", "Flat Map");

        for (String s : sentences) {
            String[] words = s.split(" ");
            for (String word : words) {
                System.out.println("Word: " + word);
            }
        }
    }
}
```

---

12 flatMap() expand collections + forEach() Logging

```
import java.util.*;

public class FlatMapExpandForEachLogging {
   public static void main(String[] args) {
      Map<String, List<String>> data = new HashMap<String, List<String>>();
      data.put("A", Arrays.asList("x", "y"));
      data.put("B", Arrays.asList("z"));

      for (List<String> values : data.values()) {
         for (String v : values) {
            System.out.println("Value: " + v);
         }
      }
   }
}
```

---

13 sorted() natural ordering + forEach() Logging

```
import java.util.*;

public class SortedNaturalForEachLogging {
   public static void main(String[] args) {
      List<Integer> numbers = new ArrayList<Integer>(
            Arrays.asList(5, 1, 3)
      );

      Collections.sort(numbers);

      for (Integer n : numbers) {
         System.out.println("Sorted: " + n);
      }
   }
}
```

---

14 sorted() custom comparator + forEach() Logging

```
import java.util.*;

public class SortedCustomForEachLogging {
   public static void main(String[] args) {
      List<String> names = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit")
      );

      Collections.sort(names, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                                    // reverse
         }
      });

      for (String name : names) {
         System.out.println("Name: " + name);
      }
   }
}
```

---

15 sorted() multi-field + forEach() Logging

```
import java.util.*;

public class SortedMultiFieldForEachLogging {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = new ArrayList<Employee>();
      employees.add(new Employee("Anurag", 30));
      employees.add(new Employee("Anurag", 25));
      employees.add(new Employee("Ravi", 35));

      Collections.sort(employees, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) {
               return nameCompare;
            }
            return e1.age - e2.age;
         }
      });

      for (Employee e : employees) {
         System.out.println(e.name + " - " + e.age);
      }
   }
}
```

16 peek() logging + forEach() Logging

```
import java.util.*;

public class PeekLoggingForEachLogging {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1, 2, 3);

      for (Integer n : list) {
         System.out.println("Peek log: " + n);
         System.out.println("Final: " + n);
      }
   }
}
```

---

17 peek() debugging + forEach() Logging

```
import java.util.*;

public class PeekDebugForEachLogging {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(10, 20, 30);

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint hit: " + n);
         }
         System.out.println("Processed: " + n);
      }
   }
}
```

---

18 peek() metrics + forEach() Logging

```
import java.util.*;

public class PeekMetricsForEachLogging {
   public static void main(String[] args) {
```

```
    List<Integer> list = Arrays.asList(5, 10, 15);
    int counter = 0;

    for (Integer n : list) {
      counter++;
      System.out.println("Value: " + n);
    }

    System.out.println("Total processed: " + counter);
  }
}
```

---

05 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() printing

filter() null checks + foreach() printing

filter() comparisons + foreach() printing

filter() method calls returning boolean + foreach() printing

distinct() rely on equals() / hashcode() + foreach() printing

distinct() immutable objects + foreach() printing

map() transform objects + foreach() printing

map() create new objects + foreach() printing

map() call pure functions + foreach() printing

flatmap() return stream + foreach() printing

flatmap() split data + foreach() printing

flatmap() expand collections + foreach() printing

sorted() natural ordering + foreach() printing

sorted() custom comparator + foreach() printing

sorted() multi-field compare + foreach() printing

peek() logging + foreach() printing

peek() debugging + foreach() printing

peek() metrics + foreach() printing

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **One clear intermediate operation** is demonstrated
- **forEach() is the terminal operation**
- **Purpose = Printing output (not logging, not collecting)**
- Java **8+ compatible**
- Clean, interview-ready, minimal noise

---

1 filter() boolean checks + forEach() Printing

```
import java.util.*;

public class FilterBooleanForEachPrint {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(10);
      numbers.add(15);
      numbers.add(20);
      numbers.add(25);

      for (Integer n : numbers) {
         if (n % 2 == 0) {
            System.out.println(n);
         }
      }
   }
}
```

---

2 filter() null checks + forEach() Printing

```
import java.util.*;

public class FilterNullForEachPrint {
```

```
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add(null);
        names.add("Amit");

        for (String name : names) {
            if (name != null) {
                System.out.println(name);
            }
        }
    }
}
```

---

3 filter() comparisons + forEach() Printing

```
import java.util.*;

public class FilterComparisonForEachPrint {
    public static void main(String[] args) {

        List<Integer> salaries = new ArrayList<Integer>();
        salaries.add(20000);
        salaries.add(40000);
        salaries.add(60000);

        for (Integer s : salaries) {
            if (s >= 30000) {
                System.out.println(s);
            }
        }
    }
}
```

---

4 filter() method call returning boolean + forEach() Printing

```
import java.util.*;

public class FilterMethodForEachPrint {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {

      List<User> users = new ArrayList<User>();
      users.add(new User("Anurag", true));
      users.add(new User("Ravi", false));

      for (User u : users) {
         if (u.isActive()) {
            System.out.println(u.name);
         }
      }
   }
}
```

---

5 distinct() rely on equals() / hashCode() + forEach() Printing

```
import java.util.*;

public class DistinctEqualsForEachPrint {

   static class Person {
      String name;

      Person(String name) {
         this.name = name;
      }

      public boolean equals(Object o) {
         if (o instanceof Person) {
            Person p = (Person) o;
            return name.equals(p.name);
         }
         return false;
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {

      List<Person> people = new ArrayList<Person>();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set<Person> unique = new HashSet<Person>(people);

      for (Person p : unique) {
         System.out.println(p.name);
      }
   }
```

```
}
```

---

6 distinct() immutable objects + forEach() Printing

```
import java.util.*;

public class DistinctImmutableForEachPrint {
   public static void main(String[] args) {

      List<String> languages = new ArrayList<String>();
      languages.add("Java");
      languages.add("Java");
      languages.add("Python");

      Set<String> unique = new HashSet<String>(languages);

      for (String lang : unique) {
         System.out.println(lang);
      }
   }
}
```

---

7 map() transform objects + forEach() Printing

```
import java.util.*;

public class MapTransformForEachPrint {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("anurag");
      names.add("amit");

      for (String name : names) {
         System.out.println(name.toUpperCase());
      }
   }
}
```

---

8 map() create new objects + forEach() Printing

```
import java.util.*;

public class MapNewObjectForEachPrint {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Amit");

      for (String name : names) {
         UserDTO dto = new UserDTO(name);
         System.out.println(dto.name);
      }
   }
}
```

---

9 map() call pure functions + forEach() Printing

```
import java.util.*;

public class MapPureFunctionForEachPrint {

   static int square(int n) {
      return n * n;
   }
```

```
    public static void main(String[] args) {

        List<Integer> numbers = new ArrayList<Integer>();
        numbers.add(2);
        numbers.add(3);
        numbers.add(4);

        for (Integer n : numbers) {
            System.out.println(square(n));
        }
    }
}
```

---

10 flatMap() return Stream + forEach() Printing

```
import java.util.*;

public class FlatMapStreamForEachPrint {
    public static void main(String[] args) {

        List<List<Integer>> lists = new ArrayList<List<Integer>>();

        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        for (List<Integer> inner : lists) {
            for (Integer n : inner) {
                System.out.println(n);
            }
        }
    }
}
```

---

11 flatMap() split data + forEach() Printing

```
import java.util.*;

public class FlatMapSplitForEachPrint {
    public static void main(String[] args) {

        List<String> sentences = new ArrayList<String>();
        sentences.add("Java Stream");
        sentences.add("Flat Map");

        for (String sentence : sentences) {
            String[] words = sentence.split(" ");
```

```
        for (String word : words) {
            System.out.println(word);
        }
    }
  }
}
```

---

12 flatMap() expand collections + forEach() Printing

```
import java.util.*;

public class FlatMapExpandForEachPrint {
  public static void main(String[] args) {

    Map<String, List<String>> data = new HashMap<String, List<String>>();
    data.put("A", Arrays.asList("x", "y"));
    data.put("B", Arrays.asList("z"));

    for (List<String> list : data.values()) {
        for (String value : list) {
            System.out.println(value);
        }
    }
  }
}
```

---

13 sorted() natural ordering + forEach() Printing

```
import java.util.*;

public class SortedNaturalForEachPrint {
  public static void main(String[] args) {

    List<Integer> numbers = new ArrayList<Integer>();
    numbers.add(5);
    numbers.add(1);
    numbers.add(3);

    Collections.sort(numbers);

    for (Integer n : numbers) {
        System.out.println(n);
    }
  }
```

```
}
```

---

14 sorted() custom comparator + forEach() Printing

```
import java.util.*;

public class SortedCustomForEachPrint {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Ravi");
      names.add("Amit");

      Collections.sort(names, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);                    // reverse
         }
      });

      for (String name : names) {
         System.out.println(name);
      }
   }
}
```

---

15 sorted() multi-field compare + forEach() Printing

```
import java.util.*;

public class SortedMultiFieldForEachPrint {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> employees = new ArrayList<Employee>();
      employees.add(new Employee("Anurag", 30));
      employees.add(new Employee("Anurag", 25));
      employees.add(new Employee("Ravi", 35));

      Collections.sort(employees, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {

            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) {
               return nameCompare;
            }

            return e1.age - e2.age;
         }
      });

      for (Employee e : employees) {
         System.out.println(e.name + " " + e.age);
      }
   }
```

```
}
```

---

16 peek() logging + forEach() Printing

```
import java.util.*;

public class PeekLoggingForEachPrint {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      for (Integer n : list) {
         System.out.println("Peek: " + n);
         System.out.println(n);
      }
   }
}
```

---

17 peek() debugging + forEach() Printing

```
import java.util.*;

public class PeekDebugForEachPrint {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      for (Integer n : list) {

         if (n == 20) {
            System.out.println("Debug hit");
         }

         System.out.println(n);
      }
   }
}
```

---

18 peek() metrics + forEach() Printing

```
import java.util.*;

public class PeekMetricsForEachPrint {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);
      int counter = 0;

      for (Integer n : list) {
         counter++;
         System.out.println(n);
      }

      System.out.println("Total processed: " + counter);
   }
}
```

---

06 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() updating external systems

filter() null checks + foreach() updating external systems

filter() comparisons + foreach() updating external systems

filter() method calls returning boolean + foreach() updating external systems

distinct() rely on equals() / hashcode() + foreach() updating external systems

distinct() immutable objects + foreach() updating external systems

map() transform objects + foreach() updating external systems

map() create new objects + foreach() updating external systems

map() call pure functions + foreach() updating external systems

flatmap() return stream + foreach() updating external systems

flatmap() split data + foreach() updating external systems

flatmap() expand collections + foreach() updating external systems

sorted() natural ordering + foreach() updating external systems

sorted() custom comparator + foreach() updating external systems

sorted() multi-field compare + foreach() updating external systems

peek() logging + foreach() updating external systems

peek() debugging + foreach() updating external systems

peek() metrics + foreach() updating external systems

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** showing **side-effects with forEach()** where the stream **updates an external system** (simulated as DB / cache / message queue / API).

 Important concept reminder (intentional here):
Using forEach() to update external systems is a **side-effect** and must be done **only at the terminal operation**. These examples are **educational and realistic**, not advocating misuse.

All examples:

- Java **8+**
- One intermediate concept per class
- External system = simulated service (ExternalSystem.update(...))

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

1 filter() boolean checks + forEach() Updating external systems

```
import java.util.*;
public class FilterBooleanUpdateExternal {
  public static void main(String[] args) {
    List<Integer> orders = Arrays.asList(100, 150, 200);

    for (Integer o : orders) {
      if (o >= 150) {
        ExternalSystem.update("Approved order: " + o);
      }
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

2 filter() null checks + forEach() Updating external systems

```
import java.util.*;

public class FilterNullUpdateExternal {
   public static void main(String[] args) {
      List<String> users = Arrays.asList("Anurag", null, "Amit");

      for (String u : users) {
         if (u != null) {
            ExternalSystem.update("User synced: " + u);
         }
      }
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

3 filter() comparisons + forEach() Updating external systems

```
import java.util.*;

public class FilterComparisonUpdateExternal {
   public static void main(String[] args) {
      List<Integer> balances = Arrays.asList(500, 2000, 7000);

      for (Integer b : balances) {
         if (b > 1000) {
            ExternalSystem.update("Premium balance: " + b);
         }
```

```
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

4 filter() method call returning boolean + forEach() Updating external systems

```
import java.util.*;

public class FilterMethodUpdateExternal {

  static class Account {
    String id;
    boolean active;

    Account(String id, boolean active) {
      this.id = id;
      this.active = active;
    }

    boolean isActive() {
      return active;
    }
  }

  public static void main(String[] args) {
    List<Account> accounts = Arrays.asList(
        new Account("A1", true),
        new Account("A2", false)
    );

    for (Account a : accounts) {
      if (a.isActive()) {
        ExternalSystem.update("Activated account: " + a.id);
      }
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

5 distinct() equals/hashCode + forEach() Updating external systems

```
import java.util.*;

public class DistinctEqualsUpdateExternal {

  static class Customer {
    String email;

    Customer(String email) {
      this.email = email;
    }

    @Override
    public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Customer)) return false;
      Customer c = (Customer) o;
      return email.equals(c.email);
    }

    @Override
    public int hashCode() {
      return email.hashCode();
    }
  }

  public static void main(String[] args) {
    List<Customer> customers = Arrays.asList(
        new Customer("a@mail.com"),
        new Customer("a@mail.com"),
        new Customer("b@mail.com")
    );

    Set<Customer> unique = new HashSet<Customer>(customers);

    for (Customer c : unique) {
      ExternalSystem.update("Customer saved: " + c.email);
```

```
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

6 distinct() immutable objects + forEach() Updating external systems

```
import java.util.*;

public class DistinctImmutableUpdateExternal {
  public static void main(String[] args) {
    List<String> tags = Arrays.asList("java", "java", "spring");

    Set<String> unique = new HashSet<String>(tags);

    for (String t : unique) {
      ExternalSystem.update("Tag indexed: " + t);
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

7 map() transform objects + forEach() Updating external systems

```
import java.util.*;

public class MapTransformUpdateExternal {
  public static void main(String[] args) {
    List<String> names = Arrays.asList("anurag", "amit");

    for (String name : names) {
      String upper = name.toUpperCase();
      ExternalSystem.update("Normalized name: " + upper);
```

```
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

8 map() create new objects + forEach() Updating external systems

```
import java.util.*;

public class MapNewObjectUpdateExternal {

  static class UserDTO {
    String name;
    UserDTO(String name) { this.name = name; }
  }

  public static void main(String[] args) {
    List<String> names = Arrays.asList("Anurag", "Amit");

    for (String name : names) {
      UserDTO dto = new UserDTO(name);
      ExternalSystem.update("DTO persisted: " + dto.name);
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

9 map() call pure functions + forEach() Updating external systems

```
import java.util.*;

public class MapPureFunctionUpdateExternal {

  static int calculateScore(int n) {
```

```
        return n * 10;
    }

    public static void main(String[] args) {
        List<Integer> inputs = Arrays.asList(1, 2, 3);

        for (Integer n : inputs) {
            int score = calculateScore(n);
            ExternalSystem.update("Score sent: " + score);
        }
    }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

10 flatMap() return Stream + forEach() Updating external systems

```
import java.util.*;

public class FlatMapStreamUpdateExternal {
    public static void main(String[] args) {
        List<List<String>> batches = new ArrayList<List<String>>();
        batches.add(Arrays.asList("A", "B"));
        batches.add(Arrays.asList("C"));

        for (List<String> batch : batches) {
            for (String v : batch) {
                ExternalSystem.update("Batch item: " + v);
            }
        }
    }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

11 flatMap() split data + forEach() Updating external systems

```
import java.util.*;

public class FlatMapSplitUpdateExternal {
   public static void main(String[] args) {
      List<String> lines = Arrays.asList("Java Stream", "Flat Map");

      for (String line : lines) {
         String[] words = line.split(" ");
         for (String word : words) {
            ExternalSystem.update("Word indexed: " + word);
         }
      }
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

12 flatMap() expand collections + forEach() Updating external systems

```
import java.util.*;

public class FlatMapExpandUpdateExternal {
  public static void main(String[] args) {

    Map<String, List<String>> data = new HashMap<String, List<String>>();
    data.put("A", Arrays.asList("1", "2"));
    data.put("B", Arrays.asList("3"));

    for (List<String> values : data.values()) {
      for (String v : values) {
        ExternalSystem.update("Value synced: " + v);
      }
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

13 sorted() natural ordering + forEach() Updating external systems

```
import java.util.*;

public class SortedNaturalUpdateExternal {
   public static void main(String[] args) {
      List<Integer> priorities = Arrays.asList(3, 1, 2);

      Collections.sort(priorities);

      for (Integer p : priorities) {
         ExternalSystem.update("Processed priority: " + p);
      }
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
		public static void update(String message) {
			System.out.println("External system updated: " + message);
		}
}
```

(Each class below assumes this helper exists in the same file or package.)

14 sorted() custom comparator + forEach() Updating external systems

```
import java.util.*;

public class SortedCustomUpdateExternal {
   public static void main(String[] args) {
      List<String> jobs = Arrays.asList("LOW", "HIGH", "MEDIUM");

      Collections.sort(jobs, Collections.reverseOrder());

      for (String j : jobs) {
         ExternalSystem.update("Job executed: " + j);
      }
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

15 sorted() multi-field compare + forEach() Updating external systems

```
import java.util.*;

public class SortedMultiFieldUpdateExternal {

  static class Task {
    String name;
    int priority;

    Task(String name, int priority) {
      this.name = name;
      this.priority = priority;
    }
  }

  public static void main(String[] args) {
    List<Task> tasks = Arrays.asList(
        new Task("Deploy", 2),
        new Task("Build", 1),
        new Task("Test", 2)
    );

    Collections.sort(tasks, new Comparator<Task>() {
      public int compare(Task t1, Task t2) {
        if (t1.priority != t2.priority) {
          return t1.priority - t2.priority;
        }
        return t1.name.compareTo(t2.name);
      }
```

```
    });

    for (Task t : tasks) {
        ExternalSystem.update("Task executed: " + t.name);
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

16 peek() logging + forEach() Updating external systems

```
import java.util.*;

public class PeekLoggingUpdateExternal {
  public static void main(String[] args) {
    List<Integer> list = Arrays.asList(1, 2, 3);

    for (Integer n : list) {
        System.out.println("Peek log: " + n);
        ExternalSystem.update("Final update: " + n);
    }
  }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
        public static void update(String message) {
                System.out.println("External system updated: " + message);
        }
}
```

(Each class below assumes this helper exists in the same file or package.)

17 peek() debugging + forEach() Updating external systems

```
import java.util.*;
```

```
public class PeekDebugUpdateExternal {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(10, 20, 30);

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint hit");
         }
         ExternalSystem.update("Processed value: " + n);
      }
   }
}
```

Shared helper (used conceptually in all examples)

```
public class ExternalSystem {
		public static void update(String message) {
			System.out.println("External system updated: " + message);
		}
}
```

(Each class below assumes this helper exists in the same file or package.)

18 peek() metrics + forEach() Updating external systems

```
import java.util.*;

public class PeekMetricsUpdateExternal {
   public static void main(String[] args) {

      int counter = 0;
      List<Integer> list = Arrays.asList(5, 10, 15);

      for (Integer n : list) {
         counter++;
         ExternalSystem.update("Metric sent: " + n);
      }

      ExternalSystem.update("Total processed: " + counter);
   }
}
```

---

07 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() combine values

filter() null checks + reduce() combine values

filter() comparisons + reduce() combine values

filter() method calls returning boolean + reduce() combine values

distinct() rely on equals() / hashcode() + reduce() combine values

distinct() immutable objects + reduce() combine values

map() transform objects + reduce() combine values

map() create new objects + reduce() combine values

map() call pure functions + reduce() combine values

flatmap() return stream + reduce() combine values

flatmap() split data + reduce() combine values

flatmap() expand collections + reduce() combine values

sorted() natural ordering + reduce() combine values

sorted() custom comparator + reduce() combine values

sorted() multi-field compare + reduce() combine values

peek() logging + reduce() combine values

peek() debugging + reduce() combine values

peek() metrics + reduce() combine values

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **every requested Stream combination**, where:

- **One primary intermediate operation** is demonstrated
- **reduce() is the terminal operation**
- **Goal = combine values** (sum, concat, merge)
- Java **8+ compatible**
- Clean, deterministic, interview-ready
- No collectors, no forEach

 Mental model
reduce(identity, accumulator) ⇒ **Combine stream elements into ONE result**

---

1 filter() boolean checks + reduce() Combine values

```
import java.util.*;

public class FilterBooleanReduce {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(10);
      numbers.add(15);
      numbers.add(20);
      numbers.add(25);

      int sum = 0;

      for (Integer n : numbers) {
         if (n % 2 == 0) {                                  // filter
            sum += n;                                       // reduce
         }
      }
```

```
        System.out.println(sum);
    }
}
```

---

2 filter() null checks + reduce() Combine values

```
import java.util.*;

public class FilterNullReduce {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add(null);
        names.add("Amit");

        String combined = "";

        for (String name : names) {
            if (name != null) {                          // filter
                combined += name;                        // reduce
            }
        }

        System.out.println(combined);
    }
}
```

---

3 filter() comparisons + reduce() Combine values

```
import java.util.*;

public class FilterComparisonReduce {
    public static void main(String[] args) {

        List<Integer> salaries = new ArrayList<Integer>();
        salaries.add(20000);
        salaries.add(40000);
```

```
        salaries.add(60000);

        int total = 0;

        for (Integer s : salaries) {
            if (s >= 30000) {
                total += s;
            }
        }

        System.out.println(total);
    }
}
```

---

4 filter() method call boolean + reduce() Combine values

```
import java.util.*;

public class FilterMethodReduce {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List<User> users = new ArrayList<User>();
        users.add(new User("Anurag", true));
        users.add(new User("Amit", true));
        users.add(new User("Ravi", false));

        int count = 0;

        for (User u : users) {
            if (u.isActive()) {
                count += 1;
            }
        }
```

```
        System.out.println(count);
    }
}
```

---

5 distinct() equals/hashCode + reduce() Combine values

```
import java.util.*;

public class DistinctEqualsReduce {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Person)) return false;
            Person p = (Person) o;
            return name.equals(p.name);
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> people = new ArrayList<Person>();
        people.add(new Person("Anurag"));
        people.add(new Person("Anurag"));
        people.add(new Person("Ravi"));

        Set<Person> unique = new HashSet<Person>(people);  // distinct

        int totalLength = 0;
```

```
        for (Person p : unique) {
            totalLength += p.name.length();
        }

        System.out.println(totalLength);
    }
}
```

---

6 distinct() immutable objects + reduce() Combine values

```
import java.util.*;

public class DistinctImmutableReduce {
    public static void main(String[] args) {

        List<String> values = new ArrayList<String>();
        values.add("Java");
        values.add("Java");
        values.add("Go");

        Set<String> unique = new HashSet<String>(values);

        int lengthSum = 0;

        for (String s : unique) {
            lengthSum += s.length();
        }

        System.out.println(lengthSum);
    }
}
```

---

7 map() transform objects + reduce() Combine values

```
import java.util.*;

public class MapTransformReduce {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("anurag");
        names.add("amit");
```

```
    int totalChars = 0;

    for (String name : names) {
        String upper = name.toUpperCase();                    // map
        totalChars += upper.length();                         // reduce
    }

    System.out.println(totalChars);
  }
}
```

---

8 map() create new objects + reduce() Combine values

```
import java.util.*;

public class MapNewObjectReduce {

  static class UserDTO {
    String name;
    UserDTO(String name) {
        this.name = name;
    }
  }

  public static void main(String[] args) {

    List<String> names = new ArrayList<String>();
    names.add("Anurag");
    names.add("Amit");

    int count = 0;

    for (String name : names) {
        UserDTO dto = new UserDTO(name);                      // map
        count += 1;                                           // reduce
    }

    System.out.println(count);
  }
}
```

---

9 map() pure functions + reduce() Combine values

```
import java.util.*;

public class MapPureFunctionReduce {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(2);
      numbers.add(3);
      numbers.add(4);

      int sumOfSquares = 0;

      for (Integer n : numbers) {
         sumOfSquares += square(n);
      }

      System.out.println(sumOfSquares);
   }
}
```

---

10 flatMap() return Stream + reduce() Combine values

```
import java.util.*;

public class FlatMapStreamReduce {
   public static void main(String[] args) {
```

```
        List<List<Integer>> lists = new ArrayList<List<Integer>>();

        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        int total = 0;

        for (List<Integer> inner : lists) {
            for (Integer n : inner) {
                total += n;
            }
        }

        System.out.println(total);
    }
}
```

---

11 flatMap() split data + reduce() Combine values

```
import java.util.*;

public class FlatMapSplitReduce {
    public static void main(String[] args) {

        List<String> sentences = new ArrayList<String>();
        sentences.add("Java Stream");
        sentences.add("Flat Map");

        int wordCount = 0;

        for (String s : sentences) {
            String[] words = s.split(" ");
            wordCount += words.length;
        }

        System.out.println(wordCount);
    }
}
```

---

12 flatMap() expand collections + reduce() Combine values

```
import java.util.*;

public class FlatMapExpandReduce {
    public static void main(String[] args) {
```

```
    Map<String, List<Integer>> data = new HashMap<String, List<Integer>>();
    data.put("A", Arrays.asList(1, 2));
    data.put("B", Arrays.asList(3));

    int sum = 0;

    for (List<Integer> list : data.values()) {
      for (Integer n : list) {
        sum += n;
      }
    }

    System.out.println(sum);
  }
}
```

---

13 sorted() natural ordering + reduce() Combine values

```
import java.util.*;

public class SortedNaturalReduce {
  public static void main(String[] args) {

    List<Integer> numbers = new ArrayList<Integer>();
    numbers.add(5);
    numbers.add(1);
    numbers.add(3);

    Collections.sort(numbers);                    // sorted

    int total = 0;

    for (Integer n : numbers) {
      total += n;
    }

    System.out.println(total);
  }
}
```

---

14 sorted() custom comparator + reduce() Combine values

```
import java.util.*;

public class SortedCustomReduce {
  public static void main(String[] args) {
```

```
        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Ravi");
        names.add("Amit");

        Collections.sort(names, Collections.reverseOrder());

        String combined = "";

        for (String name : names) {
            combined += name;
        }

        System.out.println(combined);
    }
}
```

---

15 sorted() multi-field compare + reduce() Combine values

```
import java.util.*;

public class SortedMultiFieldReduce {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<Employee>();
        employees.add(new Employee("Anurag", 30));
        employees.add(new Employee("Anurag", 25));
        employees.add(new Employee("Ravi", 35));

        Collections.sort(employees, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) return nameCompare;
                return e1.age - e2.age;
            }
        });
```

```
        int ageSum = 0;

        for (Employee e : employees) {
            ageSum += e.age;
        }

        System.out.println(ageSum);
    }
}
```

---

16 peek() logging + reduce() Combine values

```
import java.util.*;

public class PeekLoggingReduce {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);

        int sum = 0;

        for (Integer n : list) {
            System.out.println("Peek: " + n);
            sum += n;
        }

        System.out.println(sum);
    }
}
```

---

17 peek() debugging + reduce() Combine values

```
import java.util.*;

public class PeekDebugReduce {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(10, 20, 30);

        int sum = 0;

        for (Integer n : list) {
            if (n == 20) {
```

```
            System.out.println("Debug hit");
        }
        sum += n;
    }

    System.out.println(sum);
  }
}
```

---

18 peek() metrics + reduce() Combine values

```
import java.util.*;

public class PeekMetricsReduce {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(5, 10, 15);

    int sum = 0;
    int counter = 0;

    for (Integer n : list) {
        counter++;
        sum += n;
    }

    System.out.println("Sum: " + sum);
    System.out.println("Processed: " + counter);
  }
}
```

---

08 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + reduce() aggregate (sum, max, concat)

filter() null checks + reduce() aggregate (sum, max, concat)

filter() comparisons + reduce() aggregate (sum, max, concat)

filter() method calls returning boolean + reduce() aggregate (sum, max, concat)

distinct() rely on equals() / hashcode() + reduce() aggregate (sum, max, concat)

distinct() immutable objects + reduce() aggregate (sum, max, concat)

map() transform objects + reduce() aggregate (sum, max, concat)

map() create new objects + reduce() aggregate (sum, max, concat)

map() call pure functions + reduce() aggregate (sum, max, concat)

flatmap() return stream + reduce() aggregate (sum, max, concat)

flatmap() split data + reduce() aggregate (sum, max, concat)

flatmap() expand collections + reduce() aggregate (sum, max, concat)

sorted() natural ordering + reduce() aggregate (sum, max, concat)

sorted() custom comparator + reduce() aggregate (sum, max, concat)

sorted() multi-field compare + reduce() aggregate (sum, max, concat)

peek() logging + reduce() aggregate (sum, max, concat)

peek() debugging + reduce() aggregate (sum, max, concat)

peek() metrics + reduce() aggregate (sum, max, concat)

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where the **terminal operation is reduce()** and it demonstrates **aggregation**:

- **SUM** (numeric aggregation)
- **MAX** (numeric aggregation)
- **CONCAT** (string aggregation)

---

1 filter() boolean checks + reduce() Aggregate

```
import java.util.*;

public class FilterBooleanReduceAggregate {
   public static void main(String[] args) {

      List<Integer> nums = new ArrayList<Integer>();
      nums.add(10);
      nums.add(15);
      nums.add(20);

      int sum = 0;
      int max = 0;
      String concat = "";

      for (Integer n : nums) {
         if (n % 2 == 0) {
            sum += n;
            if (n > max) {
               max = n;
            }
            concat += String.valueOf(n);
         }
      }
```

```
        System.out.println(sum + " | " + max + " | " + concat);
    }
}
```

---

2 filter() null checks + reduce() Aggregate

```
import java.util.*;

public class FilterNullReduceAggregate {
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add(null);
        names.add("Amit");

        String concat = "";
        int maxLength = 0;

        for (String name : names) {
            if (name != null) {
                concat += name;
                if (name.length() > maxLength) {
                    maxLength = name.length();
                }
            }
        }

        System.out.println(concat + " | " + maxLength);
    }
}
```

---

3 filter() comparisons + reduce() Aggregate

```
import java.util.*;

public class FilterComparisonReduceAggregate {
```

```
    public static void main(String[] args) {

        List<Integer> salaries = new ArrayList<Integer>();
        salaries.add(20000);
        salaries.add(40000);
        salaries.add(60000);

        int sum = 0;
        int max = 0;

        for (Integer s : salaries) {
            if (s >= 30000) {
                sum += s;
                if (s > max) {
                    max = s;
                }
            }
        }

        System.out.println(sum + " | " + max);
    }
}
```

---

4 filter() method call boolean + reduce() Aggregate

```
import java.util.*;

public class FilterMethodReduceAggregate {

    static class User {
        String name;
        boolean active;

        User(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        boolean isActive() {
            return active;
        }
    }

    public static void main(String[] args) {

        List<User> users = new ArrayList<User>();
        users.add(new User("Anurag", true));
        users.add(new User("Amit", true));
```

```
        users.add(new User("Ravi", false));

        int count = 0;
        String concat = "";

        for (User u : users) {
            if (u.isActive()) {
                count++;
                concat += u.name;
            }
        }

        System.out.println(count + " | " + concat);
    }
}
```

---

5 distinct() equals/hashCode + reduce() Aggregate

```
import java.util.*;

public class DistinctEqualsReduceAggregate {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        public boolean equals(Object o) {
            if (o instanceof Person) {
                Person p = (Person) o;
                return name.equals(p.name);
            }
            return false;
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> people = new ArrayList<Person>();
        people.add(new Person("Anurag"));
        people.add(new Person("Anurag"));
        people.add(new Person("Ravi"));
```

```
        Set<Person> unique = new HashSet<Person>(people);

        String concat = "";
        for (Person p : unique) {
            concat += p.name;
        }

        System.out.println(concat);
    }
}
```

---

6 distinct() immutable objects + reduce() Aggregate

```
import java.util.*;

public class DistinctImmutableReduceAggregate {
    public static void main(String[] args) {

        List<String> langs = new ArrayList<String>();
        langs.add("Java");
        langs.add("Java");
        langs.add("Go");

        Set<String> unique = new HashSet<String>(langs);

        String concat = "";
        int maxLen = 0;

        for (String s : unique) {
            concat += s;
            if (s.length() > maxLen) {
                maxLen = s.length();
            }
        }

        System.out.println(concat + " | " + maxLen);
    }
}
```

---

7 map() transform objects + reduce() Aggregate

```
import java.util.*;

public class MapTransformReduceAggregate {
```

```
    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("anurag");
        names.add("amit");

        String concat = "";
        int maxLen = 0;

        for (String name : names) {
            String upper = name.toUpperCase();
            concat += upper;

            if (upper.length() > maxLen) {
                maxLen = upper.length();
            }
        }

        System.out.println(concat + " | " + maxLen);
    }
}
```

---

8 map() create new objects + reduce() Aggregate

```
import java.util.*;

public class MapNewObjectReduceAggregate {

    static class UserDTO {
        String name;
        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {

        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Amit");

        String concat = "";

        for (String name : names) {
            UserDTO dto = new UserDTO(name);
            concat += dto.name;
        }
```

```
        System.out.println(concat);
    }
}
```

---

9 map() pure functions + reduce() Aggregate

```
import java.util.*;

public class MapPureFunctionReduceAggregate {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {

        List<Integer> nums = new ArrayList<Integer>();
        nums.add(2);
        nums.add(3);
        nums.add(4);

        int sum = 0;
        int max = 0;

        for (Integer n : nums) {
            int sq = square(n);
            sum += sq;

            if (sq > max) {
                max = sq;
            }
        }

        System.out.println(sum + " | " + max);
    }
}
```

---

10 flatMap() return Stream + reduce() Aggregate

```
import java.util.*;

public class FlatMapStreamReduceAggregate {
   public static void main(String[] args) {

      List<List<Integer>> lists = new ArrayList<List<Integer>>();

      List<Integer> l1 = new ArrayList<Integer>();
      l1.add(1);
      l1.add(2);

      List<Integer> l2 = new ArrayList<Integer>();
      l2.add(3);
      l2.add(4);

      lists.add(l1);
      lists.add(l2);

      int sum = 0;

      for (List<Integer> list : lists) {
         for (Integer n : list) {
            sum += n;
         }
      }

      System.out.println(sum);
   }
}
```

---

11 flatMap() split data + reduce() Aggregate

```
import java.util.*;

public class FlatMapSplitReduceAggregate {
   public static void main(String[] args) {

      List<String> lines = new ArrayList<String>();
      lines.add("Java Stream");
      lines.add("Flat Map");

      String concat = "";

      for (String line : lines) {
         String[] words = line.split(" ");
         for (String word : words) {
            concat += word;
         }
      }

      System.out.println(concat);
   }
}
```

---

12 flatMap() expand collections + reduce() Aggregate

```
import java.util.*;

public class FlatMapExpandReduceAggregate {
   public static void main(String[] args) {

      Map<String, List<Integer>> map = new HashMap<String, List<Integer>>();

      List<Integer> listA = new ArrayList<Integer>();
      listA.add(1);
      listA.add(2);

      List<Integer> listB = new ArrayList<Integer>();
      listB.add(3);

      map.put("A", listA);
      map.put("B", listB);

      int sum = 0;

      for (List<Integer> values : map.values()) {
         for (Integer n : values) {
            sum += n;
         }
```

```
        }

        System.out.println(sum);
    }
}
```

---

13 sorted() natural ordering + reduce() Aggregate

```
import java.util.*;

public class SortedNaturalReduceAggregate {
    public static void main(String[] args) {

        List<Integer> nums = new ArrayList<Integer>();
        nums.add(5);
        nums.add(1);
        nums.add(3);

        Collections.sort(nums);

        int max = 0;

        for (Integer n : nums) {
            if (n > max) {
                max = n;
            }
        }

        System.out.println(max);
    }
}
```

---

14 sorted() custom comparator + reduce() Aggregate

```
import java.util.*;

public class SortedCustomReduceAggregate {
    public static void main(String[] args) {
```

```
        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Ravi");
        names.add("Amit");

        Collections.sort(names, new Comparator<String>() {
            public int compare(String s1, String s2) {
                return s2.compareTo(s1);                    // reverse order
            }
        });

        String concat = "";

        for (String name : names) {
            concat += name;
        }

        System.out.println(concat);
    }
}
```

---

15 sorted() multi-field compare + reduce() Aggregate

```
import java.util.*;

public class SortedMultiFieldReduceAggregate {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> emps = new ArrayList<Employee>();
        emps.add(new Employee("Anurag", 30));
        emps.add(new Employee("Anurag", 25));
        emps.add(new Employee("Ravi", 35));

        Collections.sort(emps, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) {
```

```
            return nameCompare;
        }
        return e1.age - e2.age;
    }
});

int maxAge = 0;
for (Employee e : emps) {
    if (e.age > maxAge) {
        maxAge = e.age;
    }
}

System.out.println(maxAge);
}
}
```

---

16 peek() logging + reduce() Aggregate

```
import java.util.*;

public class PeekLoggingReduceAggregate {
    public static void main(String[] args) {

        List<Integer> nums = new ArrayList<Integer>();
        nums.add(1);
        nums.add(2);
        nums.add(3);

        int sum = 0;

        for (Integer n : nums) {
            System.out.println("Peek: " + n);
            sum += n;
        }

        System.out.println(sum);
    }
}
```

---

17 peek() debugging + reduce() Aggregate

```
import java.util.*;

public class PeekDebugReduceAggregate {
    public static void main(String[] args) {
```

```
    List<Integer> nums = new ArrayList<Integer>();
    nums.add(10);
    nums.add(20);
    nums.add(30);

    int max = 0;

    for (Integer n : nums) {
      if (n == 20) {
        System.out.println("Debug hit");
      }

      if (n > max) {
        max = n;
      }
    }

    System.out.println(max);
  }
}
```

---

18 peek() metrics + reduce() Aggregate

```
import java.util.*;

public class PeekMetricsReduceAggregate {
  public static void main(String[] args) {

    List<Integer> nums = new ArrayList<Integer>();
    nums.add(5);
    nums.add(10);
    nums.add(15);

    int counter = 0;
    int sum = 0;

    for (Integer n : nums) {
      counter++;                                    // metric tracking
      sum += n;
    }

    System.out.println("Sum: " + sum);
    System.out.println("Processed: " + counter);
  }
}
```

---

09 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + count() no custom logic allowed

filter() null checks + count() no custom logic allowed

filter() comparisons + count() no custom logic allowed

filter() method calls returning boolean + count() no custom logic allowed

distinct() rely on equals() / hashcode() + count() no custom logic allowed

distinct() immutable objects + count() no custom logic allowed

map() transform objects + count() no custom logic allowed

map() create new objects + count() no custom logic allowed

map() call pure functions + count() no custom logic allowed

flatmap() return stream + count() no custom logic allowed

flatmap() split data + count() no custom logic allowed

flatmap() expand collections + count() no custom logic allowed

sorted() natural ordering + count() no custom logic allowed

sorted() custom comparator + count() no custom logic allowed

sorted() multi-field compare + count() no custom logic allowed

peek() logging + count() no custom logic allowed

peek() debugging + count() no custom logic allowed

peek() metrics + count() no custom logic allowed

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = count()**
- **No custom aggregation / reduce / collect logic**
- Only **built-in Stream operations**
- Java **8+ compatible**
- One clear intent per class

✅ "No custom logic allowed" respected:
count() is the **only terminal**, no reducers, no collectors, no mutation.

---

1 filter() boolean checks + count()

```
import java.util.*;

public class FilterBooleanCount {
   public static void main(String[] args) {
      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(10);
      numbers.add(15);
      numbers.add(20);
      numbers.add(25);

      long count = 0;

      for (Integer n : numbers) {
         if (n % 2 == 0) {
            count++;
         }
      }
```

```
        System.out.println(count);
    }
}
```

---

2 filter() null checks + count()

```
import java.util.*;

public class FilterNullCount {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Anurag", null, "Amit");

        long count = 0;

        for (String name : names) {
            if (name != null) {
                count++;
            }
        }

        System.out.println(count);
    }
}
```

---

3 filter() comparisons + count()

```
import java.util.*;

public class FilterComparisonCount {
    public static void main(String[] args) {
        List<Integer> salaries = new ArrayList<Integer>();
        salaries.add(20000);
        salaries.add(40000);
        salaries.add(60000);

        long count = 0;
```

```
        for (Integer s : salaries) {
            if (s >= 30000) {
                count++;
            }
        }

        System.out.println(count);
    }
}
```

---

4 filter() method call returning boolean + count()

```
import java.util.*;

public class FilterMethodCount {

    static class User {
        boolean active;
        User(boolean active) { this.active = active; }
        boolean isActive() { return active; }
    }

    public static void main(String[] args) {
        List<User> users = new ArrayList<User>();
        users.add(new User(true));
        users.add(new User(false));
        users.add(new User(true));

        long count = 0;

        for (User user : users) {
            if (user.isActive()) {
                count++;
            }
        }

        System.out.println(count);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + count()

```
import java.util.*;

public class DistinctEqualsCount {

   static class Person {
      String name;

      Person(String name) { this.name = name; }

      public boolean equals(Object o) {
         if (this == o) return true;
         if (!(o instanceof Person)) return false;

         Person p = (Person) o;
         return name.equals(p.name);
      }

      public int hashCode() {
         return name.hashCode();
      }
   }

   public static void main(String[] args) {
      List<Person> people = new ArrayList<Person>();
      people.add(new Person("Anurag"));
      people.add(new Person("Anurag"));
      people.add(new Person("Ravi"));

      Set<Person> unique = new HashSet<Person>(people);

      System.out.println(unique.size());
   }
```

```
}
```

---

6 distinct() immutable objects + count()

```
import java.util.*;

public class DistinctImmutableCount {
   public static void main(String[] args) {
      List<String> values = new ArrayList<String>();
      values.add("Java");
      values.add("Java");
      values.add("Python");

      Set<String> unique = new HashSet<String>(values);
      System.out.println(unique.size());
   }
}
```

---

7 map() transform objects + count()

```
import java.util.*;

public class MapTransformCount {
   public static void main(String[] args) {
      List<String> names = new ArrayList<String>();
      names.add("anurag");
      names.add("amit");

      long count = 0;

      for (String name : names) {
         String upper = name.toUpperCase();
         count++;
      }
```

```
        System.out.println(count);
    }
}
```

---

8 map() create new objects + count()

```
import java.util.*;

public class MapCreateObjectCount {

    static class UserDTO {
        String name;
        UserDTO(String name) { this.name = name; }
    }

    public static void main(String[] args) {
        List<String> names = new ArrayList<String>();
        names.add("Anurag");
        names.add("Amit");

        long count = 0;

        for (String name : names) {
            UserDTO dto = new UserDTO(name);
            count++;
        }

        System.out.println(count);
    }
}
```

---

9 map() call pure functions + count()

```
import java.util.*;

public class MapPureFunctionCount {
```

```
    static int square(int n) { return n * n; }

    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<Integer>();
        numbers.add(2);
        numbers.add(3);
        numbers.add(4);

        long count = 0;

        for (Integer n : numbers) {
            int result = square(n);
            count++;
        }

        System.out.println(count);
    }
}
```

---

10 flatMap() return Stream + count()

```
import java.util.*;

public class FlatMapStreamCount {
    public static void main(String[] args) {

        List<List<Integer>> lists = new ArrayList<List<Integer>>();

        lists.add(Arrays.asList(1, 2));
        lists.add(Arrays.asList(3, 4));

        long count = 0;

        for (List<Integer> list : lists) {
            for (Integer value : list) {
                count++;
            }
        }

        System.out.println(count);
    }
}
```

---

11 flatMap() split data + count()

```
import java.util.*;

public class FlatMapSplitCount {
   public static void main(String[] args) {

      List<String> lines = new ArrayList<String>();
      lines.add("Java Stream");
      lines.add("Flat Map");

      long count = 0;

      for (String line : lines) {
         String[] words = line.split(" ");
         for (int i = 0; i < words.length; i++) {
            count++;
         }
      }

      System.out.println(count);
   }
}
```

---

12 flatMap() expand collections + count()

```
import java.util.*;

public class FlatMapExpandCount {
   public static void main(String[] args) {

      Map<String, List<String>> data = new HashMap<String, List<String>>();
      data.put("A", Arrays.asList("1", "2"));
      data.put("B", Arrays.asList("3"));

      long count = 0;

      for (List<String> list : data.values()) {
         for (String value : list) {
            count++;
         }
      }

      System.out.println(count);
   }
}
```

---

13 sorted() natural ordering + count()

```
import java.util.*;

public class SortedNaturalCount {
   public static void main(String[] args) {

      List<Integer> numbers = new ArrayList<Integer>();
      numbers.add(5);
      numbers.add(1);
      numbers.add(3);

      Collections.sort(numbers);

      System.out.println(numbers.size());
   }
}
```

---

14 sorted() custom comparator + count()

```
import java.util.*;

public class SortedCustomCount {
   public static void main(String[] args) {

      List<String> names = new ArrayList<String>();
      names.add("Anurag");
      names.add("Ravi");
      names.add("Amit");

      Collections.sort(names, Collections.reverseOrder());

      System.out.println(names.size());
   }
}
```

---

15 sorted() multi-field compare + count()

```
import java.util.*;

public class SortedMultiFieldCount {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
```

```
        this.name = name;
        this.age = age;
    }
  }

  public static void main(String[] args) {

    List<Employee> emps = new ArrayList<Employee>();
    emps.add(new Employee("Anurag", 30));
    emps.add(new Employee("Anurag", 25));
    emps.add(new Employee("Ravi", 35));

    Collections.sort(emps, new Comparator<Employee>() {
      public int compare(Employee e1, Employee e2) {
        int nameCompare = e1.name.compareTo(e2.name);
        if (nameCompare != 0) {
          return nameCompare;
        }
        return e1.age - e2.age;
      }
    });

    System.out.println(emps.size());
  }
}
```

---

16 peek() logging + count()

```
import java.util.*;

public class PeekLoggingCount {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(1, 2, 3);

    long count = 0;

    for (Integer n : list) {
      System.out.println("Peek: " + n);
      count++;
    }

    System.out.println(count);
  }
}
```

---

17 peek() debugging + count()

```
import java.util.*;

public class PeekDebugCount {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      long count = 0;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug hit");
         }
         count++;
      }

      System.out.println(count);
   }
}
```

---

18 peek() metrics + count()

```
import java.util.*;

public class PeekMetricsCount {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);

      int metricCounter = 0;
      long count = 0;

      for (Integer n : list) {
         metricCounter++;
         count++;
      }

      System.out.println("Stream count: " + count);
      System.out.println("Metric count: " + metricCounter);
   }
}
```

---

10 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + findfirst() conditional check before first match

filter() null checks + findfirst() conditional check before first match

filter() comparisons + findfirst() conditional check before first match

filter() method calls returning boolean + findfirst() conditional check before first match

distinct() rely on equals() / hashcode() + findfirst() conditional check before first match

distinct() immutable objects + findfirst() conditional check before first match

map() transform objects + findfirst() conditional check before first match

map() create new objects + findfirst() conditional check before first match

map() call pure functions + findfirst() conditional check before first match

flatmap() return stream + findfirst() conditional check before first match

flatmap() split data + findfirst() conditional check before first match

flatmap() expand collections + findfirst() conditional check before first match

sorted() natural ordering + findfirst() conditional check before first match

sorted() custom comparator + findfirst() conditional check before first match

sorted() multi-field compare + findfirst() conditional check before first match

peek() logging + findfirst() conditional check before first match

peek() debugging + findfirst() conditional check before first match

peek() metrics + findfirst() conditional check before first match

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = findFirst()**
- Purpose = **conditional check before the first matching element**
- Demonstrates **short-circuiting** (stream stops after first match)
- Java **8+ compatible**
- No collectors, no reduce, no extra logic

 findFirst() returns Optional<T> and respects **encounter order**.

---

1 filter() boolean checks + findFirst()

```
import java.util.*;

public class FilterBooleanFindFirst {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(11, 14, 18);
      Integer firstEven = null;

      for (Integer n : list) {
         if (n % 2 == 0) {
            firstEven = n;
            break;
         }
      }

      System.out.println(firstEven != null ? firstEven : -1);
```

```
    }
}
```

---

2 filter() null checks + findFirst()

```
import java.util.*;

public class FilterNullFindFirst {
    public static void main(String[] args) {

        List<String> list = Arrays.asList(null, "Anurag", "Amit");
        String firstNonNull = null;

        for (String s : list) {
            if (s != null) {
                firstNonNull = s;
                break;
            }
        }

        System.out.println(firstNonNull != null ? firstNonNull : "NONE");
    }
}
```

---

3 filter() comparisons + findFirst()

```
import java.util.*;

public class FilterComparisonFindFirst {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(20000, 45000, 60000);
        Integer firstHigh = null;
```

```
    for (Integer s : list) {
        if (s >= 40000) {
            firstHigh = s;
            break;
        }
    }

    System.out.println(firstHigh != null ? firstHigh : 0);
  }
}
```

---

4 filter() method call returning boolean + findFirst()

```
import java.util.*;

public class FilterMethodFindFirst {

  static class User {
    String name;
    boolean active;

    User(String name, boolean active) {
        this.name = name;
        this.active = active;
    }

    boolean isActive() {
        return active;
    }
  }

  public static void main(String[] args) {

    List<User> list = Arrays.asList(
            new User("A", false),
            new User("B", true),
            new User("C", true)
    );

    User firstActive = null;

    for (User u : list) {
        if (u.isActive()) {
```

```
            firstActive = u;
            break;
        }
    }

    System.out.println(firstActive != null ? firstActive.name : "NONE");
  }
}
```

---

5 distinct() rely on equals() / hashCode() + findFirst()

```
import java.util.*;

public class DistinctEqualsFindFirst {

  static class Person {
    String name;

    Person(String name) {
      this.name = name;
    }

    public boolean equals(Object o) {
      if (o instanceof Person) {
        Person p = (Person) o;
        return name.equals(p.name);
      }
      return false;
    }

    public int hashCode() {
      return name.hashCode();
    }
  }

  public static void main(String[] args) {

    List<Person> list = Arrays.asList(
        new Person("Anurag"),
        new Person("Anurag"),
        new Person("Ravi")
```

```
        );

        Set<Person> set = new LinkedHashSet<Person>(list);

        Person first = null;
        for (Person p : set) {
            first = p;
            break;
        }

        System.out.println(first != null ? first.name : "NONE");
    }
}
```

---

6 distinct() immutable objects + findFirst()

```
import java.util.*;

public class DistinctImmutableFindFirst {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java", "Java", "Go");
        Set<String> set = new LinkedHashSet<String>(list);

        String first = null;
        for (String s : set) {
            first = s;
            break;
        }

        System.out.println(first != null ? first : "NONE");
    }
}
```

---

7 map() transform objects + findFirst()

```
import java.util.*;

public class MapTransformFindFirst {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("anurag", "amit");
        String firstUpper = null;
```

```
        for (String s : list) {
            firstUpper = s.toUpperCase();
            break;
        }

        System.out.println(firstUpper != null ? firstUpper : "NONE");
    }
}
```

---

8 map() create new objects + findFirst()

```
import java.util.*;

public class MapCreateObjectFindFirst {

    static class UserDTO {
        String name;

        UserDTO(String name) {
            this.name = name;
        }
    }

    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");
        UserDTO first = null;

        for (String s : list) {
            first = new UserDTO(s);
            break;
        }

        System.out.println(first != null ? first.name : "NONE");
    }
}
```

---

9 map() call pure functions + findFirst()

```
import java.util.*;
```

```
public class MapPureFunctionFindFirst {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 3, 4);
      Integer first = null;

      for (Integer n : list) {
         first = square(n);
         break;
      }

      System.out.println(first != null ? first : 0);
   }
}
```

---

10 flatMap() return Stream + findFirst()

```
import java.util.*;

public class FlatMapStreamFindFirst {
   public static void main(String[] args) {

      List<List<Integer>> list = new ArrayList<List<Integer>>();
      list.add(Arrays.asList(10, 20));
      list.add(Arrays.asList(30));

      Integer first = null;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            first = n;
            break;
         }
         if (first != null) break;
      }

      System.out.println(first != null ? first : 0);
   }
}
```

---

11 flatMap() split data + findFirst()

```
import java.util.*;

public class FlatMapSplitFindFirst {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Java Stream", "Flat Map");
      String firstWord = null;

      for (String s : list) {
         String[] words = s.split(" ");
         for (int i = 0; i < words.length; i++) {
            firstWord = words[i];
            break;
         }
         if (firstWord != null) break;
      }

      System.out.println(firstWord != null ? firstWord : "NONE");
   }
}
```

---

12 flatMap() expand collections + findFirst()

```
import java.util.*;

public class FlatMapExpandFindFirst {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      String first = null;

      for (List<String> values : map.values()) {
         for (String v : values) {
            first = v;
            break;
         }
         if (first != null) break;
      }

      System.out.println(first != null ? first : "NONE");
   }
}
```

---

13 sorted() natural ordering + findFirst()

```
import java.util.*;

public class SortedNaturalFindFirst {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(Arrays.asList(5, 1, 3));
      Collections.sort(list);

      Integer first = list.size() > 0 ? list.get(0) : null;

      System.out.println(first != null ? first : 0);
   }
}
```

---

14 sorted() custom comparator + findFirst()

```
import java.util.*;

public class SortedCustomFindFirst {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(list, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);
         }
      });

      String first = list.size() > 0 ? list.get(0) : null;

      System.out.println(first != null ? first : "NONE");
   }
}
```

---

15 sorted() multi-field compare + findFirst()

```
import java.util.*;

public class SortedMultiFieldFindFirst {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));

      Collections.sort(list, new Comparator<Employee>() {
         public int compare(Employee e1, Employee e2) {
            int nameCompare = e1.name.compareTo(e2.name);
            if (nameCompare != 0) return nameCompare;
            return e1.age - e2.age;
         }
      });

      Employee first = list.size() > 0 ? list.get(0) : null;
```

```
        System.out.println(first != null ? first.name + " " + first.age : "NONE");
    }
}
```

---

16 peek() logging + findFirst()

```
import java.util.*;

public class PeekLoggingFindFirst {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);
        Integer first = null;

        for (Integer n : list) {
            System.out.println("Peek: " + n);
            first = n;
            break;
        }

        System.out.println(first != null ? first : 0);
    }
}
```

---

17 peek() debugging + findFirst()

```
import java.util.*;

public class PeekDebugFindFirst {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(10, 20, 30);
        Integer first = null;
```

```
        for (Integer n : list) {
            if (n == 20) {
                System.out.println("Debug hit");
            }
            first = n;
            break;
        }

        System.out.println(first != null ? first : 0);
    }
}
```

---

18 peek() metrics + findFirst()

```
import java.util.*;

public class PeekMetricsFindFirst {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(5, 10, 15);
        int counter = 0;
        Integer first = null;

        for (Integer n : list) {
            counter++;
            first = n;
            break;
        }

        System.out.println("First: " + (first != null ? first : 0));
        System.out.println("Processed elements: " + counter);
    }
}
```

---

11 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + allmatch() validation logic

filter() null checks + allmatch() validation logic

filter() comparisons + allmatch() validation logic

filter() method calls returning boolean + allmatch() validation logic

distinct() rely on equals() / hashcode() + allmatch() validation logic

distinct() immutable objects + allmatch() validation logic

map() transform objects + allmatch() validation logic

map() create new objects + allmatch() validation logic

map() call pure functions + allmatch() validation logic

flatmap() return stream + allmatch() validation logic

flatmap() split data + allmatch() validation logic

flatmap() expand collections + allmatch() validation logic

sorted() natural ordering + allmatch() validation logic

sorted() custom comparator + allmatch() validation logic

sorted() multi-field compare + allmatch() validation logic

peek() logging + allmatch() validation logic

peek() debugging + allmatch() validation logic

peek() metrics + allmatch() validation logic

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **validation logic** (returns true only if **all elements** satisfy the predicate)
- Demonstrates **short-circuiting** (stops on first failure)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

---

1 filter() boolean checks + allMatch() (Validation)

```
import java.util.*;

public class FilterBooleanAllMatch {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean valid = true;

      for (Integer n : list) {
         if (n > 0) {                                    // filter
            if (n % 2 != 0) {                            // allMatch
               valid = false;
               break;
            }
         }
      }
```

```
        System.out.println(valid);
    }
}
```

---

2 filter() null checks + allMatch() (Validation)

```
import java.util.*;

public class FilterNullAllMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");

        boolean valid = true;

        for (String s : list) {
            if (s != null) {                                    // filter non-null
                if (s.length() == 0) {                          // allMatch
                    valid = false;
                    break;
                }
            }
        }

        System.out.println(valid);
    }
}
```

---

3 filter() comparisons + allMatch() (Validation)

```
import java.util.*;
```

```
public class FilterComparisonAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(30, 40, 50);

      boolean valid = true;

      for (Integer age : list) {
         if (age >= 18) {                                    // filter
            if (age > 60) {                                  // allMatch
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

---

4 filter() method calls returning boolean + allMatch() (Validation)

```
import java.util.*;

public class FilterMethodAllMatch {

   static class User {
      boolean active;
      User(boolean active) { this.active = active; }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(true),
            new User(true)
      );

      boolean valid = true;

      for (User u : list) {
         if (u.isActive()) {                                 // filter
            if (!u.isActive()) {                             // allMatch
               valid = false;
               break;
            }
```

```
        }
    }

    System.out.println(valid);
  }
}
```

---

5 distinct() rely on equals() / hashCode() + allMatch() (Validation)

```
import java.util.*;

public class DistinctEqualsAllMatch {

  static class Person {
    String name;
    Person(String name) { this.name = name; }

    public boolean equals(Object o) {
      if (!(o instanceof Person)) return false;
      Person p = (Person) o;
      return name.equals(p.name);
    }

    public int hashCode() {
      return name.hashCode();
    }
  }

  public static void main(String[] args) {

    List<Person> list = Arrays.asList(
        new Person("Anurag"),
        new Person("Anurag"),
        new Person("Ravi")
    );
```

```
    Set<Person> distinct = new HashSet<Person>(list);

    boolean valid = true;

    for (Person p : distinct) {
      if (p.name.length() < 4) {
        valid = false;
        break;
      }
    }

    System.out.println(valid);
  }
}
```

---

6 distinct() immutable objects + allMatch() (Validation)

```
import java.util.*;

public class DistinctImmutableAllMatch {
  public static void main(String[] args) {

    List<String> list = Arrays.asList("Java", "Java", "Go");

    Set<String> distinct = new HashSet<String>(list);

    boolean valid = true;

    for (String s : distinct) {
      if (s.length() < 2) {
        valid = false;
        break;
      }
    }

    System.out.println(valid);
  }
}
```

---

7 map() transform objects + allMatch() (Validation)

```
import java.util.*;
```

```
public class MapTransformAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean valid = true;

      for (String s : list) {
         String upper = s.toUpperCase();                    // map

         if (!upper.equals(upper.toUpperCase())) {          // allMatch
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

8 map() create new objects + allMatch() (Validation)

```
import java.util.*;

public class MapCreateObjectAllMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean valid = true;

      for (String s : list) {
         UserDTO dto = new UserDTO(s);                      // map

         if (dto.name.length() < 4) {                       // allMatch
            valid = false;
            break;
         }
      }

      System.out.println(valid);
```

```
    }
}
```

---

9 map() call pure functions + allMatch() (Validation)

```
import java.util.*;

public class MapPureFunctionAllMatch {

    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(2, 3, 4);

        boolean valid = true;

        for (Integer n : list) {
            int squared = square(n);                    // map

            if (squared <= 0) {                         // allMatch
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

---

10 flatMap() return Stream + allMatch() (Validation)

```
import java.util.*;

public class FlatMapStreamAllMatch {
    public static void main(String[] args) {

        List<List<Integer>> list = new ArrayList<List<Integer>>();
        list.add(Arrays.asList(1, 2));
        list.add(Arrays.asList(3, 4));

        boolean valid = true;

        for (List<Integer> inner : list) {
```

```
        for (Integer n : inner) {                          // flatMap
            if (n <= 0) {                                  // allMatch
                valid = false;
                break;
            }
        }
    }

    System.out.println(valid);
  }
}
```

---

11 flatMap() split data + allMatch() (Validation)

```
import java.util.*;

public class FlatMapSplitAllMatch {
  public static void main(String[] args) {

    List<String> list = Arrays.asList("Java Stream", "Flat Map");

    boolean valid = true;

    for (String s : list) {
        String[] words = s.split(" ");                     // flatMap

        for (int i = 0; i < words.length; i++) {
            if (words[i].length() < 3) {
                valid = false;
                break;
            }
        }
    }

    System.out.println(valid);
  }
}
```

---

12 flatMap() expand collections + allMatch() (Validation)

```
import java.util.*;

public class FlatMapExpandAllMatch {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean valid = true;

      for (List<String> values : map.values()) {
         for (String v : values) {
            if (v.length() == 0) {
               valid = false;
               break;
            }
         }
      }

      System.out.println(valid);
   }
}
```

---

13 sorted() natural ordering + allMatch() (Validation)

```
import java.util.*;

public class SortedNaturalAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      Collections.sort(list);                                    // sorted

      boolean valid = true;

      for (Integer n : list) {
         if (n < 1) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

14 sorted() custom comparator + allMatch() (Validation)

```
import java.util.*;

public class SortedCustomAllMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Ravi", "Amit");

      Collections.sort(list, Collections.reverseOrder());

      boolean valid = true;

      for (String name : list) {
         if (name.length() < 4) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

15 sorted() multi-field compare + allMatch() (Validation)

```
import java.util.*;

public class SortedMultiFieldAllMatch {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = Arrays.asList(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
```

```
            new Employee("Ravi", 35)
        );

        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0)
                    return nameCompare;
                return e1.age - e2.age;
            }
        });

        boolean valid = true;

        for (Employee e : list) {
            if (e.age < 18) {
                valid = false;
                break;
            }
        }

        System.out.println(valid);
    }
}
```

---

16 peek() logging + allMatch() (Validation)

```
import java.util.*;

public class PeekLoggingAllMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);

        boolean valid = true;

        for (Integer n : list) {
            System.out.println("Checking: " + n);                    // peek

            if (n <= 0) {
                valid = false;
                break;
            }
        }
        System.out.println(valid);
    }
}
```

---

17 peek() debugging + allMatch() (Validation)

```
import java.util.*;

public class PeekDebugAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);

      boolean valid = true;

      for (Integer n : list) {
         if (n == 20)
            System.out.println("Debug hit");                    // peek

         if (n < 10) {
            valid = false;
            break;
         }
      }

      System.out.println(valid);
   }
}
```

---

18 peek() metrics + allMatch() (Validation)

```
import java.util.*;

public class PeekMetricsAllMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);

      int counter = 0;
      boolean valid = true;

      for (Integer n : list) {
         counter++;                                             // peek

         if (n <= 0) {
            valid = false;
            break;
         }
      }
```

```
        System.out.println("Valid: " + valid);
        System.out.println("Processed elements: " + counter);
    }
}
```

---

12 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + allmatch() rule checks

filter() null checks + allmatch() rule checks

filter() comparisons + allmatch() rule checks

filter() method calls returning boolean + allmatch() rule checks

distinct() rely on equals() / hashcode() + allmatch() rule checks

distinct() immutable objects + allmatch() rule checks

map() transform objects + allmatch() rule checks

map() create new objects + allmatch() rule checks

map() call pure functions + allmatch() rule checks

flatmap() return stream + allmatch() rule checks

flatmap() split data + allmatch() rule checks

flatmap() expand collections + allmatch() rule checks

sorted() natural ordering + allmatch() rule checks

sorted() custom comparator + allmatch() rule checks

sorted() multi-field compare + allmatch() rule checks

peek() logging + allmatch() rule checks

peek() debugging + allmatch() rule checks

peek() metrics + allmatch() rule checks

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = allMatch()**
- Purpose = **rule checks** (business / validation rules)
- **Short-circuiting behavior** is intentional
- Java **8+**
- No collectors, no reduce, no side-effect mutation (except peek demos)

---

1 filter() boolean checks + allMatch() — Rule checks

```
import java.util.*;

public class FilterBooleanAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n > 0) {                                    // filter
            if (n % 2 != 0) {                            // allMatch condition
               rulePassed = false;
               break;
            }
         }
```

```
        }

        System.out.println(rulePassed);
    }
}
```

---

2 filter() null checks + allMatch() — Rule checks

```
import java.util.*;

public class FilterNullAllMatchRule {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Anurag", "Amit");

        boolean rulePassed = true;

        for (String name : list) {
            if (name != null) {
                if (name.length() < 4) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

---

3 filter() comparisons + allMatch() — Rule checks

```
import java.util.*;
```

```
public class FilterComparisonAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(25, 30, 40);

      boolean rulePassed = true;

      for (Integer age : list) {
         if (age >= 18) {
            if (age > 60) {
               rulePassed = false;
               break;
            }
         }
      }
      System.out.println(rulePassed);
   }
}
```

---

4 filter() method calls returning boolean + allMatch() — Rule checks

```
import java.util.*;

public class FilterMethodAllMatchRule {

   static class User {
      boolean verified;
      User(boolean verified) { this.verified = verified; }
      boolean isVerified() { return verified; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(true),
            new User(true)
      );

      boolean rulePassed = true;

      for (User u : list) {
         if (u.isVerified()) {
            if (!u.isVerified()) {
               rulePassed = false;
               break;
            }
         }
      }
```

```
        System.out.println(rulePassed);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + allMatch() — Rule checks

```
import java.util.*;

public class DistinctEqualsAllMatchRule {

    static class Person {
        String email;

        Person(String email) { this.email = email; }

        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Person)) return false;
            Person p = (Person) o;
            return email.equals(p.email);
        }

        public int hashCode() {
            return email.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> list = Arrays.asList(
                new Person("a@mail.com"),
                new Person("a@mail.com"),
                new Person("b@mail.com")
```

```
        );

        Set<Person> unique = new HashSet<Person>(list);

        boolean rulePassed = true;

        for (Person p : unique) {
            if (!p.email.contains("@")) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

---

6 distinct() immutable objects + allMatch() — Rule checks

```
import java.util.*;

public class DistinctImmutableAllMatchRule {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java", "Java", "Python");
        Set<String> unique = new HashSet<String>(list);

        boolean rulePassed = true;

        for (String lang : unique) {
            if (lang.length() < 4) {
                rulePassed = false;
                break;
            }
        }

        System.out.println(rulePassed);
    }
}
```

---

7 map() transform objects + allMatch() — Rule checks

```
import java.util.*;
```

```
public class MapTransformAllMatchRule {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean rulePassed = true;

      for (String s : list) {
         String upper = s.toUpperCase();
         if (!upper.equals(upper.toUpperCase())) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

8 map() create new objects + allMatch() — Rule checks

```
import java.util.*;

public class MapCreateObjectAllMatchRule {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean rulePassed = true;

      for (String name : list) {
         UserDTO dto = new UserDTO(name);
         if (dto.name.length() < 4) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

9 map() call pure functions + allMatch() — Rule checks

```
import java.util.*;

public class MapPureFunctionAllMatchRule {

   static int normalize(int n) { return n * 10; }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 2, 3);

      boolean rulePassed = true;

      for (Integer n : list) {
         int value = normalize(n);
         if (value < 10) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

10 flatMap() return Stream + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapStreamAllMatchRule {
   public static void main(String[] args) {

      List<List<Integer>> list = new ArrayList<List<Integer>>();
      list.add(Arrays.asList(1,2));
      list.add(Arrays.asList(3,4));

      boolean rulePassed = true;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            if (n <= 0) {
               rulePassed = false;
               break;
            }
         }
```

```
        }

        System.out.println(rulePassed);
    }
}
```

---

11 flatMap() split data + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapSplitAllMatchRule {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java Stream", "Flat Map");

        boolean rulePassed = true;

        for (String s : list) {
            String[] words = s.split(" ");
            for (String word : words) {
                if (word.length() < 3) {
                    rulePassed = false;
                    break;
                }
            }
        }

        System.out.println(rulePassed);
    }
}
```

---

12 flatMap() expand collections + allMatch() — Rule checks

```
import java.util.*;

public class FlatMapExpandAllMatchRule {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean rulePassed = true;

      for (List<String> values : map.values()) {
         for (String v : values) {
            if (v.length() == 0) {
               rulePassed = false;
               break;
            }
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

13 sorted() natural ordering + allMatch() — Rule checks

```
import java.util.*;

public class SortedNaturalAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(Arrays.asList(1,2,3));
      Collections.sort(list);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n < 1) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

14 sorted() custom comparator + allMatch() — Rule checks

```
import java.util.*;

public class SortedCustomAllMatchRule {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit")
      );

      Collections.sort(list, new Comparator<String>() {
         public int compare(String a, String b) {
            return b.compareTo(a);
         }
      });

      boolean rulePassed = true;

      for (String name : list) {
         if (name.length() < 4) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

15 sorted() multi-field compare + allMatch() — Rule checks

```
import java.util.*;

public class SortedMultiFieldAllMatchRule {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
```

```
    List<Employee> list = new ArrayList<Employee>();
    list.add(new Employee("Anurag", 30));
    list.add(new Employee("Anurag", 25));
    list.add(new Employee("Ravi", 35));

    Collections.sort(list, new Comparator<Employee>() {
      public int compare(Employee e1, Employee e2) {
        int nameCompare = e1.name.compareTo(e2.name);
        if (nameCompare != 0) return nameCompare;
        return e1.age - e2.age;
      }
    });

    boolean rulePassed = true;

    for (Employee e : list) {
      if (e.age < 18) {
        rulePassed = false;
        break;
      }
    }

    System.out.println(rulePassed);
  }
}
```

---

16 peek() logging + allMatch() — Rule checks

```
import java.util.*;

public class PeekLoggingAllMatchRule {
  public static void main(String[] args) {

    List<Integer> list = Arrays.asList(1,2,3);

    boolean rulePassed = true;

    for (Integer n : list) {
      System.out.println("Checking: " + n);
      if (n <= 0) {
        rulePassed = false;
        break;
      }
    }

    System.out.println(rulePassed);
  }
```

```
}
```

---

17 peek() debugging + allMatch() — Rule checks

```
import java.util.*;

public class PeekDebugAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10,20,30);

      boolean rulePassed = true;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint");
         }
         if (n < 10) {
            rulePassed = false;
            break;
         }
      }

      System.out.println(rulePassed);
   }
}
```

---

18 peek() metrics + allMatch() — Rule checks

```
import java.util.*;

public class PeekMetricsAllMatchRule {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5,10,15);

      int processed = 0;
      boolean rulePassed = true;

      for (Integer n : list) {
         processed++;
         if (n <= 0) {
            rulePassed = false;
            break;
         }
      }
```

```
        System.out.println("Rule passed: " + rulePassed);
        System.out.println("Processed elements: " + processed);
    }
}
```

---

13 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + anymatch() existence checks

filter() null checks + anymatch() existence checks

filter() comparisons + anymatch() existence checks

filter() method calls returning boolean + anymatch() existence checks

distinct() rely on equals() / hashcode() + anymatch() existence checks

distinct() immutable objects + anymatch() existence checks

map() transform objects + anymatch() existence checks

map() create new objects + anymatch() existence checks

map() call pure functions + anymatch() existence checks

flatmap() return stream + anymatch() existence checks

flatmap() split data + anymatch() existence checks

flatmap() expand collections + anymatch() existence checks

sorted() natural ordering + anymatch() existence checks

sorted() custom comparator + anymatch() existence checks

sorted() multi-field compare + anymatch() existence checks

peek() logging + anymatch() existence checks

peek() debugging + anymatch() existence checks

peek() metrics + anymatch() existence checks

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = anyMatch()**
- Purpose = **existence checks** (returns true if **any element** satisfies the predicate)
- Demonstrates **short-circuiting** (stops on first match)
- Java **8+** compatible
- No collectors / reduce / custom aggregation

---

1 filter() boolean checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterBooleanAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(1, 3, 4, 7);
      boolean exists = false;

      for (Integer n : list) {
         if (n > 0 && n % 2 == 0) {
            exists = true;
            break;
         }
      }
```

```
        System.out.println(exists);
    }
}
```

---

2 filter() null checks + anyMatch() — Existence check

```
import java.util.*;

public class FilterNullAnyMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList(null, "Anurag", "Amit");
        boolean exists = false;

        for (String s : list) {
            if (s != null && s.startsWith("A")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

---

3 filter() comparisons + anyMatch() — Existence check

```
import java.util.*;

public class FilterComparisonAnyMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(20000, 45000, 60000);
        boolean exists = false;

        for (Integer s : list) {
            if (s >= 30000 && s > 50000) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

---

4 filter() method calls returning boolean + anyMatch() — Existence check

```
import java.util.*;

public class FilterMethodAnyMatch {

   static class User {
      boolean active;
      User(boolean active) { this.active = active; }
      boolean isActive() { return active; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(false),
            new User(true)
      );

      boolean exists = false;

      for (User u : list) {
         if (u.isActive() && u.isActive()) {
            exists = true;
            break;
         }
      }
```

```
        System.out.println(exists);
    }
}
```

---

5 distinct() rely on equals() / hashCode() + anyMatch() — Existence check

```
import java.util.*;

public class DistinctEqualsAnyMatch {

    static class Person {
        String name;
        Person(String name) { this.name = name; }

        public boolean equals(Object o) {
            if (!(o instanceof Person)) return false;
            Person p = (Person) o;
            return name.equals(p.name);
        }

        public int hashCode() {
            return name.hashCode();
        }
    }

    public static void main(String[] args) {

        List<Person> list = Arrays.asList(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
```

```
        );

        Set<Person> distinct = new HashSet<Person>(list);

        boolean exists = false;

        for (Person p : distinct) {
            if (p.name.equals("Ravi")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

---

6 distinct() immutable objects + anyMatch() — Existence check

```
import java.util.*;

public class DistinctImmutableAnyMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java", "Java", "Python");
        Set<String> distinct = new HashSet<String>(list);

        boolean exists = false;

        for (String lang : distinct) {
            if (lang.startsWith("P")) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

---

7 map() transform objects + anyMatch() — Existence check

```
import java.util.*;

public class MapTransformAnyMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");
      boolean exists = false;

      for (String s : list) {
         String upper = s.toUpperCase();
         if (upper.equals("AMIT")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

8 map() create new objects + anyMatch() — Existence check

```
import java.util.*;

public class MapCreateObjectAnyMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");
      boolean exists = false;

      for (String s : list) {
         UserDTO dto = new UserDTO(s);
         if (dto.name.equals("Amit")) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

9 map() call pure functions + anyMatch() — Existence check

```
import java.util.*;

public class MapPureFunctionAnyMatch {

   static int square(int n) { return n * n; }

   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 3, 4);
      boolean exists = false;

      for (Integer n : list) {
         int sq = square(n);
         if (sq == 16) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

10 flatMap() return Stream + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapStreamAnyMatch {
   public static void main(String[] args) {

      List<List<Integer>> list = Arrays.asList(
            Arrays.asList(1, 2),
            Arrays.asList(3, 4)
      );

      boolean exists = false;

      for (List<Integer> inner : list) {
         for (Integer n : inner) {
            if (n == 3) {
               exists = true;
               break;
            }
```

```
        }
    }

    System.out.println(exists);
  }
}
```

---

11 flatMap() split data + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapSplitAnyMatch {
  public static void main(String[] args) {

    List<String> list = Arrays.asList("Java Stream", "Flat Map");
    boolean exists = false;

    for (String s : list) {
      String[] parts = s.split(" ");
      for (int i = 0; i < parts.length; i++) {
        if (parts[i].equals("Stream")) {
          exists = true;
          break;
        }
      }
    }

    System.out.println(exists);
  }
}
```

---

12 flatMap() expand collections + anyMatch() — Existence check

```
import java.util.*;

public class FlatMapExpandAnyMatch {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean exists = false;

      for (List<String> values : map.values()) {
         for (String v : values) {
            if (v.equals("2")) {
               exists = true;
               break;
            }
         }
      }
      System.out.println(exists);
   }
}
```

---

13 sorted() natural ordering + anyMatch() — Existence check

```
import java.util.*;

public class SortedNaturalAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(Arrays.asList(5, 1, 3));
      Collections.sort(list);

      boolean exists = false;

      for (Integer n : list) {
         if (n == 1) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

14 sorted() custom comparator + anyMatch() — Existence check

```
import java.util.*;

public class SortedCustomAnyMatch {
   public static void main(String[] args) {

      List<String> list = new ArrayList<String>(
            Arrays.asList("Anurag", "Ravi", "Amit"));

      Collections.sort(list, Collections.reverseOrder());

      boolean exists = false;

      for (String name : list) {
         if (name.equals("Amit")) {
            exists = true;
            break;
         }
      }
      System.out.println(exists);
   }
}
```

---

15 sorted() multi-field compare + anyMatch() — Existence check

```
import java.util.*;

public class SortedMultiFieldAnyMatch {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {

      List<Employee> list = new ArrayList<Employee>();
      list.add(new Employee("Anurag", 30));
      list.add(new Employee("Anurag", 25));
      list.add(new Employee("Ravi", 35));
```

```
        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0) return nameCompare;
                return e1.age - e2.age;
            }
        });

        boolean exists = false;

        for (Employee e : list) {
            if (e.age == 25) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

---

16 peek() logging + anyMatch() — Existence check

```
import java.util.*;

public class PeekLoggingAnyMatch {
    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);
        boolean exists = false;

        for (Integer n : list) {
            System.out.println("Checking: " + n);
            if (n == 2) {
                exists = true;
                break;
            }
        }

        System.out.println(exists);
    }
}
```

---

17 peek() debugging + anyMatch() — Existence check

```
import java.util.*;
```

```
public class PeekDebugAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(10, 20, 30);
      boolean exists = false;

      for (Integer n : list) {
         if (n == 20) {
            System.out.println("Debug breakpoint");
         }
         if (n == 20) {
            exists = true;
            break;
         }
      }

      System.out.println(exists);
   }
}
```

---

18 peek() metrics + anyMatch() — Existence check

```
import java.util.*;

public class PeekMetricsAnyMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(5, 10, 15);
      int processed = 0;
      boolean exists = false;

      for (Integer n : list) {
         processed++;
         if (n == 10) {
            exists = true;
            break;
         }
      }

      System.out.println("Exists: " + exists);
      System.out.println("Processed elements: " + processed);
   }
}
```

---

14 - complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + nonematch() negative validation

filter() null checks + nonematch() negative validation

filter() comparisons + nonematch() negative validation

filter() method calls returning boolean + nonematch() negative validation

distinct() rely on equals() / hashcode() + nonematch() negative validation

distinct() immutable objects + nonematch() negative validation

map() transform objects + nonematch() negative validation

map() create new objects + nonematch() negative validation

map() call pure functions + nonematch() negative validation

flatmap() return stream + nonematch() negative validation

flatmap() split data + nonematch() negative validation

flatmap() expand collections + nonematch() negative validation

sorted() natural ordering + nonematch() negative validation

sorted() custom comparator + nonematch() negative validation

sorted() multi-field compare + nonematch() negative validation

peek() logging + nonematch() negative validation

peek() debugging + nonematch() negative validation

peek() metrics + nonematch() negative validation

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** for **each requested Stream combination**, where:

- **Terminal operation = noneMatch()**
- Purpose = **negative validation** (assert *no element* violates a rule)
- **Short-circuiting**: stops on the first violation
- Java **8+**, no collectors/reduce

---

1 filter() boolean checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterBooleanNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(2, 4, 6, 8);

      boolean ok = true;

      for (Integer n : list) {
         if (n > 0) {
            if (n % 2 != 0) {                                    // noneMatch condition
               ok = false;
               break;
            }
```

```
        }
    }

    System.out.println(ok);
  }
}
```

---

2 filter() null checks + noneMatch() — Negative validation

```
import java.util.*;

public class FilterNullNoneMatch {
  public static void main(String[] args) {

    List<String> list = Arrays.asList("Anurag", "Amit");

    boolean ok = true;

    for (String s : list) {
      if (s != null) {
        if (s.length() == 0) {                    // isEmpty()
          ok = false;
          break;
        }
      }
    }

    System.out.println(ok);
  }
}
```

---

3 filter() comparisons + noneMatch() — Negative validation

```
import java.util.*;

public class FilterComparisonNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = Arrays.asList(25, 30, 40);

      boolean ok = true;

      for (Integer age : list) {
         if (age >= 18) {
            if (age > 60) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

4 filter() method calls returning boolean + noneMatch() — Negative validation

```
import java.util.*;

public class FilterMethodNoneMatch {

   static class User {
      boolean banned;
      User(boolean banned) { this.banned = banned; }
      boolean isBanned() { return banned; }
   }

   public static void main(String[] args) {

      List<User> list = Arrays.asList(
            new User(false),
            new User(false)
      );

      boolean ok = true;

      for (User u : list) {
         if (u.isBanned()) {
            ok = false;
            break;
```

```
        }
    }

    System.out.println(ok);
  }
}
```

---

5 distinct() rely on equals() / hashCode() + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctEqualsNoneMatch {

  static class Person {
    String email;

    Person(String email) { this.email = email; }

    public boolean equals(Object o) {
      if (this == o) return true;
      if (!(o instanceof Person)) return false;
      Person p = (Person) o;
      return email.equals(p.email);
    }

    public int hashCode() {
      return email.hashCode();
    }
  }

  public static void main(String[] args) {

    List<Person> list = Arrays.asList(
        new Person("a@mail.com"),
```

```
            new Person("a@mail.com"),
            new Person("b@mail.com")
        );

        Set<Person> unique = new HashSet<Person>(list);

        boolean ok = true;

        for (Person p : unique) {
            if (p.email.endsWith("@spam.com")) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
    }
}
```

---

6 distinct() immutable objects + noneMatch() — Negative validation

```
import java.util.*;

public class DistinctImmutableNoneMatch {
    public static void main(String[] args) {

        List<String> list = Arrays.asList("Java", "Java", "Python");

        Set<String> unique = new HashSet<String>(list);

        boolean ok = true;

        for (String lang : unique) {
            if (lang.length() < 3) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
    }
}
```

---

7 map() transform objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapTransformNoneMatch {
   public static void main(String[] args) {

      List<String> list = Arrays.asList("anurag", "amit");

      boolean ok = true;

      for (String s : list) {
         String upper = s.toUpperCase();
         if (upper.indexOf(" ") != -1) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
   }
}
```

---

8 map() create new objects + noneMatch() — Negative validation

```
import java.util.*;

public class MapCreateObjectNoneMatch {

   static class UserDTO {
      String name;
      UserDTO(String name) { this.name = name; }
   }

   public static void main(String[] args) {

      List<String> list = Arrays.asList("Anurag", "Amit");

      boolean ok = true;

      for (String name : list) {
         UserDTO dto = new UserDTO(name);
         if (dto.name.trim().length() == 0) {
            ok = false;
            break;
         }
      }

      System.out.println(ok);
```

```
    }
}
```

---

9 map() call pure functions + noneMatch() — Negative validation

```
import java.util.*;

public class MapPureFunctionNoneMatch {

    static int normalize(int n) {
        return n * 10;
    }

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);

        boolean ok = true;

        for (Integer n : list) {
            int value = normalize(n);
            if (value <= 0) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
    }
}
```

---

10 flatMap() return Stream + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapStreamNoneMatch {
    public static void main(String[] args) {

        List<List<Integer>> list = Arrays.asList(
                Arrays.asList(1, 2),
                Arrays.asList(3, 4)
        );

        boolean ok = true;

        for (List<Integer> inner : list) {
```

```
        for (Integer n : inner) {
            if (n < 0) {
                ok = false;
                break;
            }
        }
    }

    System.out.println(ok);
  }
}
```

---

11 flatMap() split data + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapSplitNoneMatch {
  public static void main(String[] args) {

    List<String> list = Arrays.asList("Java Stream", "Flat Map");

    boolean ok = true;

    for (String s : list) {
        String[] words = s.split(" ");
        for (int i = 0; i < words.length; i++) {
            if (words[i].length() < 3) {
                ok = false;
                break;
            }
        }
    }

    System.out.println(ok);
  }
}
```

---

12 flatMap() expand collections + noneMatch() — Negative validation

```
import java.util.*;

public class FlatMapExpandNoneMatch {
   public static void main(String[] args) {

      Map<String, List<String>> map = new HashMap<String, List<String>>();
      map.put("A", Arrays.asList("1", "2"));
      map.put("B", Arrays.asList("3"));

      boolean ok = true;

      for (List<String> values : map.values()) {
         for (String s : values) {
            if (s.length() == 0) {
               ok = false;
               break;
            }
         }
      }

      System.out.println(ok);
   }
}
```

---

13 sorted() natural ordering + noneMatch() — Negative validation

```
import java.util.*;

public class SortedNaturalNoneMatch {
   public static void main(String[] args) {

      List<Integer> list = new ArrayList<Integer>(
            Arrays.asList(1, 2, 3)
      );

      Collections.sort(list);

      boolean ok = true;

      for (Integer n : list) {
         if (n <= 0) {
            ok = false;
            break;
         }
      }
```

```
        System.out.println(ok);
    }
}
```

---

14 sorted() custom comparator + noneMatch() — Negative validation

```
import java.util.*;

public class SortedCustomNoneMatch {
    public static void main(String[] args) {

        List<String> list = new ArrayList<String>(
                Arrays.asList("Anurag", "Ravi", "Amit")
        );

        Collections.sort(list, new Comparator<String>() {
            public int compare(String a, String b) {
                return b.compareTo(a);                    // reverse
            }
        });

        boolean ok = true;

        for (String name : list) {
            if (name.length() < 4) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
```

```
    }
}
```

---

15 sorted() multi-field compare + noneMatch() — Negative validation

```
import java.util.*;

public class SortedMultiFieldNoneMatch {

    static class Employee {
        String name;
        int age;

        Employee(String name, int age) {
            this.name = name;
            this.age = age;
        }
    }

    public static void main(String[] args) {

        List<Employee> list = new ArrayList<Employee>();
        list.add(new Employee("Anurag", 30));
        list.add(new Employee("Anurag", 25));
        list.add(new Employee("Ravi", 35));

        Collections.sort(list, new Comparator<Employee>() {
            public int compare(Employee e1, Employee e2) {
                int nameCompare = e1.name.compareTo(e2.name);
                if (nameCompare != 0)
```

```
            return nameCompare;
        return e1.age - e2.age;
    }
});

boolean ok = true;

for (Employee e : list) {
    if (e.age < 18) {
        ok = false;
        break;
    }
}

System.out.println(ok);
}
}
```

---

16 peek() logging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekLoggingNoneMatch {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(1, 2, 3);

        boolean ok = true;

        for (Integer n : list) {

            // peek equivalent (logging)
            System.out.println("Checking: " + n);

            // noneMatch condition
            if (n < 0) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
    }
}
```

---

17 peek() debugging + noneMatch() — Negative validation

```
import java.util.*;

public class PeekDebugNoneMatch {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(10, 20, 30);

        boolean ok = true;

        for (Integer n : list) {

            // peek equivalent (debugging)
            if (n == 20) {
                System.out.println("Debug hit");
            }

            // noneMatch condition
            if (n < 10) {
                ok = false;
                break;
            }
        }

        System.out.println(ok);
```

```
    }
}
```

---

18 peek() metrics + noneMatch() — Negative validation

```
import java.util.*;

public class PeekMetricsNoneMatch {

    public static void main(String[] args) {

        List<Integer> list = Arrays.asList(5, 10, 15);

        int seen = 0;                                   // AtomicInteger replaced with simple counter
        boolean ok = true;

        for (Integer n : list) {

            // peek equivalent (metrics)
            seen++;

            // noneMatch condition
            if (n <= 0) {
                ok = false;
                break;
            }
        }

        System.out.println("OK: " + ok);
        System.out.println("Processed: " + seen);
```

```
  }
}
```